Evaluator Reference — Contents

Confidential — lookup by exercise ID, matches the ID printed on each candidate page.

Evaluator Reference — Contents (cont'd)

PYTHON MEDIUM

PY-01 — Mutable Default Argument Trace

EXPECTED APPROACH

```
def append_item(item, bucket=[]):
    bucket.append(item)
    return bucket

print(append_item(1))       # [1]
print(append_item(2))       # [1, 2]
print(append_item(3, []))   # [3]
print(append_item(4))       # [1, 2, 4]
```

WHAT TO CHECK

• Candidate gives all four outputs correctly, especially that the second and fourth calls keep accumulating into the same list.
• Candidate identifies that append_item(3, []) resets to a fresh list because a new empty list literal was passed explicitly.
• Candidate correctly states that default arguments are evaluated once at function-definition time, not per call.
• Candidate does not claim each call starts from a fresh empty list.

COMMON MISTAKES

• Assuming bucket=[] creates a brand new empty list on every call, giving outputs [1], [2], [3], [4].
• Forgetting that passing [] explicitly at the third call bypasses the shared default entirely.
• Losing track of list identity and describing the bug in terms of value copying instead of object references.

SUGGESTED SCORING

Correct output for all four prints	50%
Correct explanation of shared list identity	30%
Correct explanation of one-time default evaluation	20%

PYTHON MEDIUM

PY-02 — Late-Binding Closures In A Loop

EXPECTED APPROACH

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)

results = [f() for f in funcs]
print(results)   # [2, 2, 2]
```

WHAT TO CHECK

- Candidate states the output is [2, 2, 2], not [0, 1, 2].
- Candidate explains that lambdas capture the variable i by reference, not by its value at the time the lambda was created.
- Candidate explains that by the time the lambdas are called, the loop has finished and i is 2 for all of them.
- Candidate proposes a correct fix such as lambda i=i: i (default-argument capture) or wrapping in a small factory function.

COMMON MISTAKES

- Assuming each lambda captures a snapshot of i at the time it was appended, giving [0, 1, 2].
- Confusing this with list/dict comprehension scoping rules, which behave differently in Python 3.
- Proposing a fix that still references the loop variable by name without actually freezing its value at each iteration.

SUGGESTED SCORING

Correct predicted output	40%
Correct explanation of late binding	40%
Valid proposed fix	20%

PYTHON MEDIUM

PY-03 — Decorator Application Order

EXPECTED APPROACH

```
def shout(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper

def exclaim(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs) + "!"
    return wrapper

@exclaim
@shout
def greet(name):
    return f"hello {name}"

print(greet("sam"))   # HELLO SAM!
```

WHAT TO CHECK

• Candidate gives the exact output HELLO SAM!
• Candidate correctly identifies that shout (closest to the function definition) wraps greet first, and exclaim wraps that already-wrapped result.
• Candidate correctly determines that swapping the order still yields HELLO SAM!, because .upper() does not affect the punctuation character '!' regardless of whether it was appended before or after uppercasing.
• Candidate's reasoning traces through *args/**kwargs forwarding rather than guessing at the final string.

COMMON MISTAKES

• Assuming decorators apply top-to-bottom on the call rather than bottom-up on the definition, reversing the wrap order in their explanation.
• Assuming swapping decorator order must change the output, without checking that '!' is case-invariant under .upper().
• Not tracing through *args/**kwargs forwarding and assuming the wrappers alter the input arguments rather than the return value.

SUGGESTED SCORING

Correct output string	35%
Correct wrap order reasoning	35%
Correct reasoning about swapped-order equivalence	30%

PYTHON MEDIUM

PY-04 — String Slicing Edge Cases

EXPECTED APPROACH

```
s = "engineering"
print(s[::-1])     # gnireenigne
print(s[2:8:2])    # gne
print(s[-4:])       # ring
print(s[100:])      # (empty string)
```

WHAT TO CHECK

- Candidate correctly gives 'gnireenigne' for the full reverse.
- Candidate correctly gives 'gne' for s[2:8:2], correctly stepping by 2 starting at index 2.
- Candidate correctly gives 'ring' for s[-4:], correctly counting from the end.
- Candidate correctly states s[100:] returns an empty string rather than raising an error, and explains that slice bounds are clamped to the string's length instead of validated strictly like index access.

COMMON MISTAKES

- Miscounting the step-2 slice and including or excluding one extra character.
- Confusing negative indexing direction and getting the wrong last-4-characters substring.
- Assuming s[100:] raises an IndexError, confusing slicing semantics with single-index access semantics (s[100] would indeed raise IndexError).

SUGGESTED SCORING

All four outputs correct	70%
Correct explanation of out-of-range slice clamping	30%

PYTHON MEDIUM

PY-05 — Generator Exhaustion Behavior

EXPECTED APPROACH

```
def counter(n):
    for i in range(n):
        yield i

gen = counter(3)
print(list(gen))        # [0, 1, 2]
print(list(gen))        # []
print(sum(counter(3)))  # 3
```

WHAT TO CHECK

• Candidate gives [0, 1, 2], then [], then 3, in that order.
• Candidate explains that a generator object is a single-use iterator that remembers its position and raises StopIteration once exhausted.
• Candidate explains that counter(3) on the third line creates a brand-new, independent generator object, distinct from gen.

COMMON MISTAKES

• Assuming list(gen) can be called repeatedly to get the same values each time, like a list.
• Confusing the generator function counter with the generator object it returns when called.
• Believing sum(counter(3)) reuses the already-exhausted gen instance instead of creating a new one.

SUGGESTED SCORING

Correct three outputs	45%
Correct explanation of exhaustion	35%
Correct explanation of fresh generator on third call	20%

PYTHON MEDIUM

PY-06 — Leaky Shared Default Dictionary

EXPECTED APPROACH

```
def group_by_length(words, groups=None):
    if groups is None:
        groups = {}
    for w in words:
        groups.setdefault(len(w), []).append(w)
    return groups
```

WHAT TO CHECK

• Candidate correctly explains that {} in the default is created once at function-definition time and reused across every call that doesn't pass groups explicitly.
• Fixed code uses groups=None as the default and creates a new dict inside the function body when groups is None.
• Fixed code still allows an explicit dictionary to be passed in and accumulated into, preserving the opt-in accumulation feature.
• Candidate demonstrates or explains that group_by_length(['cat','dog']) followed by group_by_length(['ox']) now gives independent results: {3: ['cat','dog']} and {2: ['ox']}.

COMMON MISTAKES

• Fixing it by moving the dict creation to a local variable named groups without changing the default value from {} to None, which does not fix the bug.
• Removing the ability to pass in a custom dict at all, over-fixing the function's interface.
• Explaining the bug as 'Python caches function results' rather than correctly describing default-argument evaluation timing.

SUGGESTED SCORING

Correct diagnosis of shared default	35%
Correct fix using None sentinel	45%
Preserves optional explicit dict parameter	20%

PYTHON MEDIUM

PY-07 — Off-By-One Adjacent Comparison

EXPECTED APPROACH

```
def is_sorted_ascending(nums):
    for i in range(len(nums) - 1):
        if nums[i] > nums[i + 1]:
            return False
    return True
```

WHAT TO CHECK

- Candidate identifies an IndexError on the nums[i] > nums[i + 1] line, occurring when i equals len(nums) - 1 (the last valid index), because nums[i + 1] then goes out of range.
- Fixed code loops over range(len(nums) - 1) instead of range(len(nums)).
- Fixed code returns True for the empty list and for single-element lists without raising an error.
- Candidate does not introduce a new off-by-one in the other direction, e.g. skipping a valid comparison.

COMMON MISTAKES

- Fixing it by adding a try/except around the comparison instead of correcting the loop bound.
- Changing range(len(nums)) to range(len(nums) - 2), which skips a valid comparison.
- Not testing the fix against edge cases like [] or a single-element list.

SUGGESTED SCORING

Correct bug diagnosis (IndexError, exact cause)	35%
Correct fixed loop bound	45%
Handles empty/single-element edge cases	20%

PYTHON MEDIUM

PY-08 — Unbound Local Total Accumulator

EXPECTED APPROACH

```
def make_accumulator():
    total = 0
    def add_to_total(x):
        nonlocal total
        total += x
        return total
    return add_to_total
```

WHAT TO CHECK

- Candidate identifies UnboundLocalError and explains that total += x is treated as an assignment to total, which makes Python treat total as a local variable throughout the whole function body, so it is read before being assigned.
- Candidate does not simply patch it with 'global total' as the only answer; credit for also explaining why a closure-based design avoids shared mutable module state, if offered.
- Redesigned solution uses nonlocal inside a nested function backed by an enclosing factory function.
- Candidate demonstrates two sequential calls whose return values correctly accumulate, e.g. 5 then 8.

COMMON MISTAKES

- Assuming the error is a NameError rather than an UnboundLocalError.
- Fixing it only by adding 'global total', without recognizing this still leaves a shared mutable global that any caller could stomp on.
- Forgetting the nonlocal keyword in the nested function, which raises the same class of scoping error again.

SUGGESTED SCORING

Correct exception + scoping explanation	35%
Working closure-based redesign with nonlocal	45%
Correct demonstration across multiple calls	20%

PYTHON MEDIUM

PY-09 — Missing Base Case In Recursive Flatten

EXPECTED APPROACH

```
def flatten_sum(data):
    if not data:
        return 0
    if isinstance(data[0], list):
        return flatten_sum(data[0]) + flatten_sum(data[1:])
    else:
        return data[0] + flatten_sum(data[1:])
```

WHAT TO CHECK

- Candidate explains that repeated slicing with data[1:] eventually produces an empty list, and data[0] on an empty list raises IndexError because there is no base case for that scenario.
- Fixed code adds an explicit base case (e.g. if not data: return 0) checked before indexing into data.
- Fixed code still returns the correct total, 21, for the given nested example.
- Candidate correctly states flatten_sum([]) should return 0, matching the identity element for addition.

COMMON MISTAKES

- Wrapping the indexing in a try/except IndexError instead of adding a proper base case, which can mask other real bugs.
- Adding the base-case check after the isinstance check, so data[0] is still evaluated on an empty list first.
- Getting the recursive sum wrong after adding the fix, e.g. an off-by-one in the slice, so the total no longer equals 21.

SUGGESTED SCORING

Correct bug diagnosis	30%
Correct base case fix	40%
Correct final sum + empty-list behavior	30%

PYTHON MEDIUM

PY-10 — Deleting From A Dict While Iterating

EXPECTED APPROACH

```
def remove_expired(inventory):
    for key in list(inventory.keys()):
        if inventory[key]["expired"]:
            del inventory[key]
    return inventory
```

WHAT TO CHECK

- Candidate identifies RuntimeError: dictionary changed size during iteration, and explains that deleting a key while the dict's own iterator is active invalidates that iterator.
- Fixed code iterates over a separate snapshot of the keys (e.g. list(inventory.keys())) rather than the live dict or its .items() view.
- Fixed code correctly leaves only {'b': {'expired': False}} in the resulting dictionary for the given example.
- Candidate does not propose catching and ignoring the RuntimeError as the fix.

COMMON MISTAKES

- Believing the bug only shows up 'sometimes' rather than understanding it is deterministic given the dict's internal size-change check.
- Building a fix around a while loop that still mutates the same dict being iterated, reintroducing the same problem.
- Iterating over inventory.values() instead of keys, losing the ability to call del inventory[key].

SUGGESTED SCORING

Correct exception + cause explanation	35%
Correct snapshot-based fix	45%
Correct final output for given input	20%

PYTHON MEDIUM

PY-11 — Why Generators Save Memory

EXPECTED APPROACH

```
def read_large_file_lines(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip()
```

WHAT TO CHECK

• Candidate explains that a generator produces one line at a time on demand instead of materializing the entire file's lines in memory at once.
• Candidate correctly states that the file handle stays open only while the generator is actively being consumed (the with block is part of the generator's paused frame) and closes once exhausted or garbage collected.
• Candidate correctly explains that re-iterating an already-exhausted generator yields nothing; it does not restart from the beginning.
• Candidate names a real downside, such as not knowing the length up front, not being able to index into it, or only being able to consume it once.

COMMON MISTAKES

• Claiming the generator loads the whole file at once 'lazily' but still all in memory, missing the point of streaming.
• Assuming the generator can be iterated multiple times like a list.
• Not connecting the with block's lifetime to the generator's paused/resumed execution model.

SUGGESTED SCORING

Correct memory/streaming explanation	45%
Correct single-use/exhaustion explanation	35%
Valid named downside	20%

PYTHON MEDIUM

PY-12 — Set-Based Duplicate Detection

EXPECTED APPROACH

```
def find_duplicates(items):
    seen = set()
    duplicates = set()
    for item in items:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return duplicates
```

WHAT TO CHECK

• Candidate explains that set membership testing (item in seen) is average O(1) versus O(n) for a list, making the whole function average O(n) instead of O(n^2).
• Candidate notes the returned set has no guaranteed order and cannot itself contain duplicate entries, unlike a list which could hold the same value twice.
• Candidate correctly explains that unhashable elements like lists cannot be added to a set and would raise a TypeError, requiring a different data structure or a hashable proxy such as tuples.

COMMON MISTAKES

• Saying sets are faster 'because they're built-in' without mentioning hashing or average O(1) lookup.
• Assuming the returned duplicates set preserves the original order or count of duplicate occurrences.
• Not realizing unhashable items would crash the function at seen.add(item) or duplicates.add(item).

SUGGESTED SCORING

Correct time-complexity explanation	45%
Correct explanation of set output properties	30%
Correct reasoning about unhashable elements	25%

PYTHON MEDIUM

PY-13 — How lru_cache Speeds Up Fibonacci

EXPECTED APPROACH

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

WHAT TO CHECK

• Candidate explains that lru_cache memoizes results keyed by call arguments, turning exponential-time naive recursion into linear time by avoiding recomputation of repeated subproblems.
• Candidate explains lru_cache stores results keyed by the (hashable) arguments, so calling fib with an unhashable argument like a list would raise a TypeError.
• Candidate explains maxsize=None means the cache can grow without bound and never evicts old entries, and identifies unbounded memory growth as the risk in a long-running process.

COMMON MISTAKES

• Saying lru_cache 'makes the function faster' without explaining the memoization/avoided-recomputation mechanism.
• Confusing lru_cache's hashability requirement with needing sortable or comparable arguments instead of hashable ones.
• Assuming maxsize=None caps the cache at some Python-internal default rather than meaning unbounded.

SUGGESTED SCORING

Correct complexity explanation	40%
Correct hashability explanation	30%
Correct maxsize/memory risk explanation	30%

PYTHON MEDIUM

PY-14 — Context Manager Exception Semantics

EXPECTED APPROACH

```
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.start
        return False
```

WHAT TO CHECK

• Candidate explains __enter__ runs before the with block's body executes, and __exit__ runs after the body finishes, whether it completed normally or via an exception.
• Candidate explains exc_type/exc_val/exc_tb carry the exception class, instance, and traceback if an exception propagated out of the block, and are all None if the block completed without raising.
• Candidate explains that returning a truthy value from __exit__ suppresses the exception so it does not propagate further, whereas returning False (or None) lets it propagate normally, meaning this Timer intentionally always lets exceptions through.

COMMON MISTAKES

• Assuming __exit__ is only called when an exception occurs, not on normal completion.
• Assuming the return value of __exit__ has no effect on exception behavior.
• Confusing exc_val, the exception instance, with exc_type, the exception class.

SUGGESTED SCORING

Correct enter/exit timing explanation	35%
Correct explanation of exc_* parameters	30%
Correct explanation of return-value suppression semantics	35%

PYTHON MEDIUM

PY-15 — Recursion Depth And Stack Cost

EXPECTED APPROACH

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

WHAT TO CHECK

• Candidate explains each recursive call adds a new stack frame, and frames aren't released until the base case is reached and the calls start returning, so stack usage grows linearly with n.
• Candidate correctly names RecursionError, tied to Python's default recursion limit (sys.getrecursionlimit(), roughly 1000).
• Candidate proposes an iterative rewrite, e.g. a for/while loop accumulating a running product, that uses constant stack space regardless of n.

COMMON MISTAKES

• Confusing this with an infinite loop rather than a bounded-but-too-deep call stack.
• Assuming Python automatically applies tail-call optimization to avoid the stack growth; CPython does not.
• Proposing to 'just increase the recursion limit' with sys.setrecursionlimit as the only fix, without acknowledging this still risks a real C stack overflow/crash rather than genuinely solving the scalability issue.

SUGGESTED SCORING

Correct call-stack growth explanation	35%
Correct exception + cause	30%
Valid iterative rewrite proposal	35%

PYTHON MEDIUM

PY-16 — Loop To List Comprehension

EXPECTED APPROACH

```
def even_squares(nums):
    return [n ** 2 for n in nums if n % 2 == 0]
```

WHAT TO CHECK

• Rewritten function uses a single-line list comprehension combining the filter condition and the transformation.
• Rewritten function produces identical output to the original for the same inputs, e.g. even_squares([1,2,3,4,5,6]) == [4, 16, 36].
• Candidate does not introduce a generator expression where a list was expected; the return type should remain a list.
• Candidate gives a reasonable justification for when the loop form is preferable, such as needing to log or debug each iteration, or the transform having side effects.

COMMON MISTAKES

• Writing the condition and transformation in the wrong syntactic order, e.g. placing if before the expression incorrectly.
• Accidentally changing the filter logic, e.g. filtering on n**2 % 2 == 0 instead of n % 2 == 0.
• Returning a generator object instead of a list, changing the function's observable behavior for callers expecting len() or repeated iteration.

SUGGESTED SCORING

Correct, equivalent comprehension	60%
Correct output preserved	25%
Reasonable trade-off answer	15%

PYTHON MEDIUM

PY-17 — Recursive Fibonacci To Iterative

EXPECTED APPROACH

```
def fib_iterative(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

WHAT TO CHECK

- Iterative version produces the same sequence as the recursive one for at least n = 0 through 9 (0,1,1,2,3,5,8,13,21,34).
- Iterative version uses O(1) extra space, a fixed number of variables, rather than building a list or recursing.
- Candidate correctly states the recursive version is exponential time, roughly O(2^n), while the iterative version is linear time, O(n).
- Candidate does not accidentally off-by-one the loop bound, e.g. fib_iterative(5) should be 5, not 8 or 3.

COMMON MISTAKES

- Swapping the tuple assignment order (writing b, a = a, b instead of a, b = b, a + b), producing an incorrect sequence.
- Using range(n + 1) or range(n - 1) instead of range(n), shifting the result by one Fibonacci index.
- Claiming both versions have the same time complexity.

SUGGESTED SCORING

Correct iterative implementation	55%
Matches recursive output exactly	25%
Correct complexity comparison	20%

PYTHON MEDIUM

PY-18 — Iterator Class To Generator Function

EXPECTED APPROACH

```
def fibonacci_gen(limit):
    a, b = 0, 1
    for _ in range(limit):
        yield a
        a, b = b, a + b
```

WHAT TO CHECK

• Generator function yields identical values to the class for the same limit, e.g. limit=8 gives [0,1,1,2,3,5,8,13].
• Generator function raises StopIteration automatically once the loop completes, without any manual raise StopIteration or explicit count tracking.
• Candidate correctly explains that a, b, and the position in the loop are preserved automatically between yield points by the generator's paused frame, replacing the class's self.a/self.b/self.count instance attributes.

COMMON MISTAKES

• Manually raising StopIteration inside the generator body, which is unnecessary and in more complex cases can even cause a RuntimeError under PEP 479.
• Forgetting to update a, b after the yield, producing an infinite generator of zeros or incorrect values.
• Using range(limit) incorrectly so the generator yields one too many or too few Fibonacci numbers.

SUGGESTED SCORING

Correct generator implementation	55%
Matches class output exactly	25%
Correct explanation of implicit state	20%

PYTHON MEDIUM

PY-19 — Nested Loops To itertools.product

EXPECTED APPROACH

```
from itertools import product

def all_pairs(list1, list2):
    return list(product(list1, list2))
```

WHAT TO CHECK

• Rewritten function returns tuples in the same order as the nested-loop version, with list1's element varying slowest and list2's fastest.
• Rewritten function actually uses itertools.product rather than reimplementing the same nested loop under a different name.
• Candidate explains that itertools.product accepts an arbitrary number of iterables, avoiding manually nesting one loop per additional input list.

COMMON MISTAKES

• Passing the two lists to product as a single argument, e.g. product([list1, list2]), instead of as separate arguments, producing the wrong result.
• Forgetting to wrap product(...) in list(...), returning an itertools.product object instead of a list of tuples.
• Getting the pair order backwards relative to the original nested loop.

SUGGESTED SCORING

Correct itertools.product usage	55%
Matches original output/order exactly	25%
Correct scaling explanation	20%

PYTHON MEDIUM

PY-20 — Manual String Concatenation To Join

EXPECTED APPROACH

```
def build_csv_row(fields):
    return ",".join(str(field) for field in fields)
```

WHAT TO CHECK

• Rewritten function uses ','.join(...) with a generator expression or list comprehension that stringifies each field.
• Rewritten function produces identical output to the original for the same input, including correctly handling non-string field values like numbers.
• Candidate explains that because strings are immutable in Python, each += creates a new string object and copies the previous contents, making naive repeated concatenation potentially O(n^2), whereas join builds the result in one pass.

COMMON MISTAKES

• Forgetting to convert non-string fields, like integers or floats, to strings before joining, causing a TypeError.
• Adding a trailing or leading comma by mishandling the join logic manually instead of trusting str.join to place separators correctly.
• Claiming += concatenation is inefficient 'because of loops' rather than because of string immutability and reallocation.

SUGGESTED SCORING

Correct join-based implementation	55%
Matches original output exactly, incl. type coercion	25%
Correct efficiency explanation	20%

PYTHON MEDIUM

PY-21 — Top-K Words With Counter

EXPECTED APPROACH

```
from collections import Counter

def top_k_words(text, k):
    words = text.lower().split()
    return Counter(words).most_common(k)
```

WHAT TO CHECK

- Solution imports and uses collections.Counter rather than a hand-rolled counting dict.
- Solution lowercases the text before splitting, so 'The' and 'the' count as the same word.
- Solution uses Counter's most_common(k) method rather than manually sorting items by count.
- For the sample text 'the cat sat on the mat the cat ran' with k=2, solution returns [('the', 3), ('cat', 2)].

COMMON MISTAKES

- Forgetting to lowercase before counting, so 'The' and 'the' are treated as different words.
- Manually sorting a dict's items() by count instead of using most_common, which is more error-prone and verbose.
- Splitting on a fixed delimiter like ',' instead of whitespace via .split().

SUGGESTED SCORING

Correct use of Counter	45%
Correct case normalization	25%
Correct output for sample input	30%

PYTHON MEDIUM

PY-22 — Grouping Records With defaultdict

EXPECTED APPROACH

```
from collections import defaultdict

def group_students_by_grade(students):
    groups = defaultdict(list)
    for name, grade in students:
        groups[grade].append(name)
    return dict(groups)
```

WHAT TO CHECK

• Solution imports and uses collections.defaultdict(list) rather than manually checking membership before appending.
• Solution converts the defaultdict back to a plain dict before returning, or the candidate explains why this matters for a caller who might rely on KeyError behavior for unknown keys.
• For students = [('Alice','A'), ('Bob','B'), ('Carol','A'), ('Dan','C')], solution returns {'A': ['Alice', 'Carol'], 'B': ['Bob'], 'C': ['Dan']}.

COMMON MISTAKES

• Using defaultdict but returning it directly without converting to dict(), silently changing behavior for later lookups on missing keys.
• Using groups.setdefault(grade, []).append(name) instead of the specifically requested defaultdict, missing the intent of the exercise.
• Losing insertion order of names within a group by using a set instead of a list.

SUGGESTED SCORING

Correct use of defaultdict	45%
Correct grouping/order behavior	35%
Sensible handling of return type (dict vs defaultdict)	20%

PYTHON MEDIUM

PY-23 — Run-Length Encoding With groupby

EXPECTED APPROACH

```
from itertools import groupby

def run_length_encode(s):
    return [(ch, len(list(group))) for ch, group in groupby(s)]
```

WHAT TO CHECK

- Solution imports and uses itertools.groupby rather than a manual index-tracking loop.
- Solution correctly converts each group iterator to a count, e.g. via len(list(group)) or sum(1 for _ in group).
- run_length_encode('aaabbbccd') returns exactly [('a', 3), ('b', 3), ('c', 2), ('d', 1)].
- Candidate correctly explains that groupby only merges adjacent equal runs, unlike Counter which would merge all occurrences regardless of position, and that sorting first would destroy the information needed for RLE.

COMMON MISTAKES

- Using collections.Counter instead of groupby, incorrectly merging non-adjacent occurrences of the same character.
- Forgetting that each group from groupby is a one-time-use iterator, and trying to measure its length without converting it to a list first.
- Sorting the string before grouping, which changes the run structure and produces the wrong encoding.

SUGGESTED SCORING

Correct use of groupby	50%
Correct handling of group iterators	25%
Correct output + explanation	25%

PYTHON MEDIUM

PY-24 — Function Composition With reduce

EXPECTED APPROACH

```
from functools import reduce

def compose(*funcs):
    return reduce(lambda f, g: lambda x: g(f(x)), funcs)
```

WHAT TO CHECK

- Solution imports and uses functools.reduce to fold the sequence of functions into one combined function.
- The combined function applies funcs in left-to-right order, first function in *funcs runs first, not right-to-left mathematical composition order.
- compose(lambda x: x + 1, lambda x: x * 2, str)(3) correctly returns the string '8', since (3+1)*2 = 8, then stringified.
- Solution handles compose() being called with an arbitrary count of functions correctly, working for 2, 3, or more.

COMMON MISTAKES

- Implementing right-to-left mathematical composition, f(g(h(x))), instead of the left-to-right pipeline order the task asked for.
- Using a mutable accumulator or for-loop that rebinds the same variable name across iterations in a way that breaks due to late-binding closures, similar to the classic loop-lambda pitfall.
- Not handling the case of a single function passed to compose gracefully.

SUGGESTED SCORING

Correct use of reduce	45%
Correct left-to-right application order	35%
Correct demonstrated output	20%

PYTHON MEDIUM

PY-25 — Validated Dataclass With Post Init

EXPECTED APPROACH

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class Product:
    name: str
    price: float
    tags: List[str] = field(default_factory=list)

    def __post_init__(self):
        if self.price < 0:
            raise ValueError("price cannot be negative")
```

WHAT TO CHECK

• Solution adds a __post_init__ method that raises ValueError (or another appropriate exception) when price is negative.
• Solution keeps field(default_factory=list) for tags rather than a bare mutable default, and candidate correctly explains this avoids the mutable-default-argument sharing bug (dataclasses actually reject tags: List[str] = [] outright at class-definition time).
• Candidate demonstrates or explains that appending to one Product's tags does not affect another Product's tags list.
• Constructing Product('Bad', -5) raises the expected exception; constructing Product('Widget', 9.99) succeeds.

COMMON MISTAKES

• Putting the validation in __init__ instead of __post_init__, which either doesn't run since dataclasses generate their own __init__, or requires manually reimplementing field assignment.
• Using tags: List[str] = [] directly as a mutable default, not realizing dataclasses actually reject this with a ValueError at class-definition time, unlike plain functions.
• Validating price in a way that also rejects zero, when only negative prices should be rejected.

SUGGESTED SCORING

Correct __post_init__ validation	45%
Correct default_factory reasoning	30%
Correct demonstration of independent tags lists	25%

PYTHON MEDIUM

PY-26 — Private State Via Closures

EXPECTED APPROACH

```
def make_counter(start=0):
    count = start
    def increment(step=1):
        nonlocal count
        count += step
        return count
    return increment
```

WHAT TO CHECK

• Solution uses a closure, an inner function referencing an enclosing-scope variable, with the nonlocal keyword to persist and mutate count between calls.
• Two independently created counters, e.g. c1 = make_counter() and c2 = make_counter(100), maintain separate counts that don't interfere with each other.
• increment accepts an optional step argument, defaulting to 1, that is added to the running count.
• Candidate correctly explains that count lives only in make_counter's local scope and is only reachable through the closure captured by increment, with no external reference to it.

COMMON MISTAKES

• Forgetting the nonlocal keyword, causing an UnboundLocalError the first time increment tries to modify count.
• Using a mutable default argument or a global variable instead of a proper closure, breaking independence between separate counters.
• Making increment take no step argument at all, missing part of the required interface.

SUGGESTED SCORING

Correct closure implementation with nonlocal	50%
Correct independent state across instances	30%
Correct explanation of privacy/scope	20%

PYTHON MEDIUM

PY-27 — Forwarding Arguments With args kwargs

EXPECTED APPROACH

```
def log_call(func):
    def wrapper(*args, **kwargs):
        wrapper.calls.append((args, kwargs))
        return func(*args, **kwargs)
    wrapper.calls = []
    return wrapper
```

WHAT TO CHECK

- wrapper accepts *args, **kwargs and forwards them unchanged to func(*args, **kwargs), so decorated functions of any signature still work correctly.
- wrapper.calls correctly accumulates (args, kwargs) tuples across multiple calls, preserving call order.
- add(1, 2) then add(1, 2, c=3) produces add.calls == [((1, 2), {}), ((1, 2), {'c': 3})], correctly splitting positional vs keyword arguments.
- Candidate explains that *args/**kwargs let the decorator work generically for any wrapped function's signature, without hardcoding parameter names.

COMMON MISTAKES

- Hardcoding the wrapped function's specific parameters instead of using *args/**kwargs, making log_call only work for functions with that exact signature.
- Forgetting to return func(*args, **kwargs)'s result from wrapper, silently breaking the decorated function's return value.
- Initializing wrapper.calls = [] inside wrapper itself, resetting the log on every call, instead of once outside at decoration time.

SUGGESTED SCORING

Correct *args/**kwargs forwarding	45%
Correct call-log accumulation	35%
Correct explanation of generality	20%

PYTHON MEDIUM

PY-28 — Filtering Dict Comprehension

EXPECTED APPROACH

```
def long_word_lengths(words, min_len):
    return {w.upper(): len(w) for w in words if len(w) > min_len}
```

WHAT TO CHECK

• Solution uses a single dict comprehension combining both the filter condition (len(word) > min_len) and the key/value transformation, rather than a loop with an if and dict[key] = value.
• Keys are uppercased words; values are the length of the original word.
• long_word_lengths(['cat','elephant','dog','giraffe'], 3) returns exactly {'ELEPHANT': 8, 'GIRAFFE': 7}.
• Candidate correctly uses strict greater-than for min_len, excluding words whose length equals min_len, matching the stated condition.

COMMON MISTAKES

• Using >= instead of > for the length filter, including words exactly at min_len.
• Computing len() on the uppercased word instead of the original; harmless here since .upper() doesn't change length, but worth checking candidate understands why this doesn't matter specifically for length.
• Building the dict correctly but via an explicit loop, missing the specific instruction to write it as a single dict comprehension.

SUGGESTED SCORING

Correct single-comprehension implementation	55%
Correct filter boundary (strict >)	25%
Correct output for sample input	20%

PYTHON MEDIUM

PY-29 — Custom Exception With Try Except Else Finally

EXPECTED APPROACH

```
class ConfigError(Exception):
    pass

def parse_positive_int(raw):
    log = []
    try:
        value = int(raw)
    except ValueError:
        raise ConfigError(f"'{raw}' is not a valid integer")
    else:
        if value <= 0:
            raise ConfigError(f"{value} is not positive")
        log.append("parsed ok")
        return value
    finally:
        log.append("attempt finished")
```

WHAT TO CHECK

• Solution defines and raises ConfigError, not a bare ValueError, for both the non-numeric and non-positive failure cases, ideally with a clear descriptive message.
• Solution uses an else clause for the post-parse positivity check, so that check only runs when int(raw) succeeded without exception, and its own error-raising logic isn't accidentally caught by the surrounding except ValueError.
• Solution uses a finally block that runs regardless of which path was taken: success, non-numeric failure, or non-positive failure.
• parse_positive_int('42') returns 42; parse_positive_int('abc') and parse_positive_int('-5') both raise ConfigError.

COMMON MISTAKES

• Putting the value <= 0 check inside the try block right after int(raw), which risks the check's own raised error being caught by the same except ValueError clause if it happened to raise ValueError instead of ConfigError.
• Catching the generic Exception instead of the specific ValueError, silently swallowing unrelated bugs.
• Forgetting the finally block entirely, or putting logging logic only on the success path so it doesn't run on failures.

SUGGESTED SCORING

Correct exception translation to ConfigError	40%
Correct use of else for post-success logic	30%
Correct use of finally for always-run logging	30%

PYTHON MEDIUM

PY-30 — Operator Overloading With Dunder Methods

EXPECTED APPROACH

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

    def __repr__(self):
        return f"Vector2D({self.x}, {self.y})"
```

WHAT TO CHECK

- __add__ returns a new Vector2D instance, not a tuple or a mutated self, with correctly summed x and y.
- __eq__ compares both x and y and returns a boolean, not accidentally comparing object identity or only one attribute.
- __repr__ returns a string in the form Vector2D(x, y) reflecting the actual attribute values, enabling readable debugging output.
- Vector2D(1,2) + Vector2D(3,4) == Vector2D(4,6) evaluates to True given a correct __add__ and __eq__.

COMMON MISTAKES

- Implementing __add__ to mutate and return self instead of constructing and returning a new Vector2D, causing unexpected aliasing bugs.
- Forgetting __eq__ entirely and relying on default identity-based comparison, which makes equal-valued vectors compare unequal.
- Implementing __str__ instead of __repr__ when repr() or interactive display is what's being exercised, or getting the format string wrong.

SUGGESTED SCORING

Correct __add__	35%
Correct __eq__	35%
Correct __repr__	30%

TYPESCRIPT MEDIUM

TS-01 — Generic TTL Cache

EXPECTED APPROACH

```
interface CacheEntry<T> {
  value: T;
  expiresAt: number;
}

class TypedCache<T> {
  private store = new Map<string, CacheEntry<T>>();

  set(key: string, value: T, ttlMs: number): void {
    this.store.set(key, { value, expiresAt: Date.now() + ttlMs });
  }

  get(key: string): T | undefined {
    const entry = this.store.get(key);
    if (!entry) return undefined;
    if (Date.now() > entry.expiresAt) {
      this.store.delete(key);
      return undefined;
    }
    return entry.value;
  }

  has(key: string): boolean {
    return this.get(key) !== undefined;
  }
}
```

WHAT TO CHECK

- Class is declared generic over `T` and the internal Map is typed with `CacheEntry<T>`, not `any`.
- `get` checks both existence and expiry before returning a value.
- Expired entries are removed from the store (not just skipped) when encountered.
- `has` does not duplicate the expiry-check logic (reuses `get` or equivalent).

COMMON MISTAKES

- Typing the store as `Map<string, T>` and storing expiry separately, losing atomicity between value and timestamp.
- Using `any` for the value type instead of the generic parameter `T`.
- Forgetting to delete expired entries, causing the map to grow unbounded.
- Comparing `expiresAt` with `<` instead of `>`, inverting the expiry logic.

SUGGESTED SCORING

Correct generic typing throughout	40%
Correct expiry logic in get/set	40%
has() implemented cleanly	20%

TYPESCRIPT MEDIUM

TS-02 — API Response Renderer

EXPECTED APPROACH

```
type ApiResponse<T> =
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };

function renderResponse<T>(response: ApiResponse<T>, formatData: (data: T) => string): string {
  switch (response.status) {
    case 'loading':
      return 'Loading...';
    case 'success':
      return formatData(response.data);
    case 'error':
      return `Error: ${response.error}`;
    default: {
      const exhaustiveCheck: never = response;
      return exhaustiveCheck;
    }
  }
}
```

WHAT TO CHECK

- Uses the `status` discriminant to narrow the union in each branch.
- Accesses `response.data` only inside the `success` branch and `response.error` only inside `error`.
- Includes an exhaustiveness check (`never`) so future added variants cause a compile error.
- Does not use type assertions (`as`) to force access to fields that don't exist on a branch.

COMMON MISTAKES

- Using `if/else` chains checking `'data' in response` instead of the discriminant field, which is fragile.
- Casting `response as any` to access `.data` regardless of status.
- Omitting the exhaustiveness (`never`) check so a new union member silently falls through.
- Forgetting the generic `<T>` and hardcoding a concrete data type.

SUGGESTED SCORING

Correct discriminated union narrowing	45%
All three states handled correctly	35%
Exhaustiveness check present	20%

TYPESCRIPT MEDIUM

TS-03 — Partial Profile Update

EXPECTED APPROACH

```
interface UserProfile {
  id: string;
  name: string;
  email: string;
  age: number;
  bio: string;
}

function updateProfile(
  profile: UserProfile,
  updates: Partial<Pick<UserProfile, 'name' | 'email' | 'age' | 'bio'>>
): UserProfile {
  return { ...profile, ...updates };
}
```

WHAT TO CHECK

- Uses `Pick` (or an equivalent explicit union of keys) to exclude `id` from the updatable fields.
- Wraps the picked type in `Partial` so any subset (including zero fields) can be passed.
- Return type remains the full, non-optional `UserProfile`.
- Implementation actually merges via spread rather than mutating the input `profile`.

COMMON MISTAKES

- Using `Partial<UserProfile>` directly, which allows `id` to be overwritten.
- Mutating `profile` in place instead of returning a new merged object.
- Making the return type `Partial<UserProfile>` instead of the full interface.
- Manually retyping name/email/age/bio instead of deriving from `UserProfile` with `Pick`.

SUGGESTED SCORING

Correct utility-type composition excluding id	45%
Correct merge implementation	35%
Return type integrity	20%

TYPESCRIPT MEDIUM

TS-04 — Retry With Backoff

EXPECTED APPROACH

```
async function retry<T>(
  fn: () => Promise<T>,
  attempts: number,
  delayMs: number
): Promise<T> {
  let lastError: unknown;
  for (let i = 0; i < attempts; i++) {
    try {
      return await fn();
    } catch (err) {
      lastError = err;
      if (i < attempts - 1) {
        await new Promise<void>((resolve) => setTimeout(resolve, delayMs));
      }
    }
  }
  throw lastError;
}
```

WHAT TO CHECK

- Function signature preserves the generic `T` all the way to the returned `Promise<T>`.
- Caught error is typed as `unknown` (not `any`), consistent with modern strict TS catch typing.
- Delay only occurs between attempts, not after the last failed attempt.
- Final failure re-throws rather than returning `undefined` or swallowing the error.

COMMON MISTAKES

- Typing the caught error as `any` or `Error` without a runtime check.
- Adding a delay after the last (final) failed attempt, wasting time needlessly.
- Returning `undefined` when all attempts are exhausted instead of throwing.
- Forgetting `await` before calling `fn()`, causing unhandled promise rejections.

SUGGESTED SCORING

Correct generic async typing	35%
Correct retry/delay control flow	40%
Proper error propagation on exhaustion	25%

TYPESCRIPT MEDIUM

TS-05 — Payment Method Narrowing

EXPECTED APPROACH

```
type PaymentMethod =
  | { type: 'credit_card'; cardNumber: string; cvv: string }
  | { type: 'paypal'; email: string }
  | { type: 'bank_transfer'; accountNumber: string; routingNumber: string };

function isCreditCard(
  method: PaymentMethod
): method is { type: 'credit_card'; cardNumber: string; cvv: string } {
  return method.type === 'credit_card';
}

function maskPaymentInfo(method: PaymentMethod): string {
  if (isCreditCard(method)) {
    return `Card ending in ${method.cardNumber.slice(-4)}`;
  }
  if (method.type === 'paypal') {
    return `PayPal: ${method.email}`;
  }
  return `Bank account ending in ${method.accountNumber.slice(-4)}`;
}
```

WHAT TO CHECK

- `isCreditCard` is written as a proper `method is {...}` type predicate function.
- Fields are accessed only after narrowing (no unchecked access to `cardNumber` on the raw union).
- All three payment method variants are handled distinctly.
- Masking logic correctly slices the last 4 characters.

COMMON MISTAKES

- Accessing `method.cardNumber` before narrowing, causing a compile error.
- Writing `isCreditCard` as a plain boolean-returning function without the `is` predicate, losing narrowing in callers.
- Using `as` casts to force the type instead of proper narrowing.
- Missing the `bank_transfer` branch and falling through incorrectly.

SUGGESTED SCORING

Correct type predicate usage	35%
Correct narrowing for all variants	40%
Correct masking logic	25%

TYPESCRIPT MEDIUM

TS-06 — Fluent Query Builder

EXPECTED APPROACH

```
interface QueryState {
  table: string;
  filters: string[];
  limitValue?: number;
}

class QueryBuilder {
  private state: QueryState;

  constructor(table: string) {
    this.state = { table, filters: [] };
  }

  where(condition: string): this {
    this.state.filters.push(condition);
    return this;
  }

  limit(n: number): this {
    this.state.limitValue = n;
    return this;
  }

  build(): string {
    let query = `SELECT * FROM ${this.state.table}`;
    if (this.state.filters.length > 0) {
      query += ` WHERE ${this.state.filters.join(' AND ')}`;
    }
    if (this.state.limitValue !== undefined) {
      query += ` LIMIT ${this.state.limitValue}`;
    }
    return query;
  }
}
```

WHAT TO CHECK

- `where` and `limit` are typed to return `this` (polymorphic self-type), not the literal class name.
- Internal mutable state is kept private and not exposed directly.
- `build` correctly joins multiple filters with `AND`.
- `build` omits the `WHERE`/`LIMIT` clauses entirely when not set, rather than emitting empty clauses.

COMMON MISTAKES

- Returning `QueryBuilder` instead of `this`, which breaks chaining correctness in subclasses.
- Making `state` public, breaking encapsulation.
- Emitting `WHERE ` with a trailing empty clause when no filters were added.
- Using `0` as a falsy check for `limitValue` instead of `!== undefined`, causing `limit(0)` to be silently dropped.

SUGGESTED SCORING

Correct chainable typing (this)	35%
Correct state accumulation	30%
Correct build() string assembly	35%

TYPESCRIPT MEDIUM

TS-07 — Strongly Typed Event Emitter

EXPECTED APPROACH

```
interface EventMap {
  login: { userId: string };
  logout: { userId: string };
  error: { message: string; code: number };
}

class TypedEmitter<T extends object> {
  private listeners: { [K in keyof T]?: Array<(payload: T[K]) => void> } = {};

  on<K extends keyof T>(event: K, handler: (payload: T[K]) => void): void {
    if (!this.listeners[event]) {
      this.listeners[event] = [];
    }
    this.listeners[event]!.push(handler);
  }

  emit<K extends keyof T>(event: K, payload: T[K]): void {
    this.listeners[event]?.forEach((handler) => handler(payload));
  }
}

const emitter = new TypedEmitter<EventMap>();
emitter.on('login', (payload) => {
  console.log(payload.userId);
});
emitter.emit('login', { userId: 'u1' });
```

WHAT TO CHECK

- `on` and `emit` are generic over `K extends keyof T`, tying the event name to its payload type.
- The listeners map is typed with a mapped type keyed by `T`, not `Record<string, Function[]>`.
- Example usage shows the handler parameter is inferred correctly without manual annotation.
- `emit` safely handles the case where no listeners are registered for an event.

COMMON MISTAKES

- Typing `handler` as `(payload: any) => void`, losing all payload safety.
- Using a plain `string` for the event name parameter instead of `K extends keyof T`.
- Forgetting the non-null assertion or initialization check before pushing into `listeners[event]`.
- Not handling the case where `listeners[event]` is undefined in `emit`, causing a runtime error.

SUGGESTED SCORING

Correct generic event/payload coupling	45%
Correct on/emit implementation	35%
Working example demonstrating inference	20%

TYPESCRIPT MEDIUM

TS-08 — Generic Stack

EXPECTED APPROACH

```
class Stack<T> {
  private items: T[] = [];

  push(item: T): void {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }

  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }

  get size(): number {
    return this.items.length;
  }

  isEmpty(): boolean {
    return this.items.length === 0;
  }
}
```

WHAT TO CHECK

- `pop` and `peek` are typed to return `T | undefined`, correctly reflecting the empty-stack case under strict null checks.
- Internal array is private and generic over `T`.
- `size` is implemented as a getter (property access, not a method call) or clearly documented as such.
- `isEmpty` correctly reflects the internal array length.

COMMON MISTAKES

- Typing `pop`/`peek` as returning `T` (non-optional), which is unsound when the stack is empty.
- Using `any[]` instead of `T[]` for internal storage.
- Implementing `size` as a regular method but calling it like a property (or vice versa) inconsistently.
- Mutating `items` directly from outside the class due to a public field.

SUGGESTED SCORING

Correct generic typing and encapsulation	40%
Correct push/pop/peek behavior	40%
size/isEmpty correctness	20%

TYPESCRIPT MEDIUM

TS-09 — Typed Config Parser

EXPECTED APPROACH

```
function parseConfigValue(value: string, type: 'number'): number;
function parseConfigValue(value: string, type: 'boolean'): boolean;
function parseConfigValue(value: string, type: 'string'): string;
function parseConfigValue(
  value: string,
  type: 'number' | 'boolean' | 'string'
): number | boolean | string {
  switch (type) {
    case 'number':
      return Number(value);
    case 'boolean':
      return value === 'true';
    case 'string':
      return value;
  }
}

const port = parseConfigValue('8080', 'number');
const enabled = parseConfigValue('true', 'boolean');
```

WHAT TO CHECK

- Three distinct overload signatures are declared above the implementation signature.
- The implementation signature's parameter/return types are a superset compatible with all overloads.
- Example call-site variables show narrowed return types (number, boolean) rather than the union.
- Switch/implementation body actually returns correctly-typed values for each branch.

COMMON MISTAKES

- Only writing the union-typed implementation signature without any overloads, forcing callers to cast the result.
- Declaring overloads but with a mismatched or incompatible implementation signature that TS rejects.
- Returning `value` (string) for the `number` case without converting via `Number()`.
- Comparing `value === true` instead of `value === 'true'` for the boolean branch (type error since value is a string).

SUGGESTED SCORING

Correct overload declarations	45%
Correct implementation signature and body	35%
Demonstrated correct inference at call sites	20%

TYPESCRIPT MEDIUM

TS-10 — Deep Readonly Settings

EXPECTED APPROACH

```
interface AppSettings {
  theme: string;
  notifications: {
    email: boolean;
    sms: boolean;
  };
}

type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends object ? DeepReadonly<T[K]> : T[K];
};

function freezeSettings(settings: AppSettings): DeepReadonly<AppSettings> {
  return settings as DeepReadonly<AppSettings>;
}

// const frozen = freezeSettings({ theme: 'dark', notifications: { email: true, sms: false } });
// frozen.notifications.email = false; // would be a compile error: Cannot assign to 'email' because it is a r
```

WHAT TO CHECK

- `DeepReadonly` is a mapped type using `[K in keyof T]` with a `readonly` modifier.
- Recursion condition correctly checks `T[K] extends object` (or similar) before recursing, falling back to `T[K]` for primitives.
- `freezeSettings` return type is `DeepReadonly<AppSettings>`, not plain `AppSettings`.
- Candidate demonstrates understanding that nested mutation is now blocked, not just top-level.

COMMON MISTAKES

- Using the built-in `Readonly<T>` directly, which does not recurse into nested objects.
- Forgetting the conditional recursion, applying `readonly` but leaving nested object types unchanged (still mutable).
- Applying `DeepReadonly` to array types incorrectly without considering array element behavior (acceptable to note as a limitation, but should not crash the type).
- Returning `settings` typed as plain `AppSettings` instead of `DeepReadonly<AppSettings>`.

SUGGESTED SCORING

Correct recursive mapped type definition	50%
Correct application in freezeSettings	30%
Correct explanation/demonstration of effect	20%

TYPESCRIPT MEDIUM

TS-11 — Async Return Type Extractor

EXPECTED APPROACH

```
type AsyncReturnType<T extends (...args: any[]) => Promise<any>> =
  T extends (...args: any[]) => Promise<infer R> ? R : never;

async function fetchUser(id: string): Promise<{ id: string; name: string }> {
  return { id, name: 'Alice' };
}

type User = AsyncReturnType<typeof fetchUser>;

function printUser(user: User): void {
  console.log(user.id, user.name);
}
```

WHAT TO CHECK

- `AsyncReturnType` uses `infer` inside a conditional type to pull out the Promise's resolved type.
- Type parameter is constrained to functions returning a `Promise` (`(...args: any[]) => Promise<any>`).
- `User` is derived via `typeof fetchUser` rather than hand-typed, proving the utility works.
- `printUser` successfully accesses `.id` and `.name` on the derived type with no casts.

COMMON MISTAKES

- Forgetting `infer` and instead writing `T extends Promise<any> ? T : never`, which doesn't unwrap function return types.
- Not constraining `T` to a function type, causing the conditional type to be inapplicable to `typeof fetchUser`.
- Manually retyping `User` as `{ id: string; name: string }` instead of deriving it, defeating the purpose of the exercise.
- Using `ReturnType<typeof fetchUser>` alone (without unwrapping), leaving `User` as `Promise<{...}>`.

SUGGESTED SCORING

Correct conditional type with infer	50%
Correct derivation via typeof	30%
Correct usage in printUser	20%

TYPESCRIPT MEDIUM

TS-12 — Order Status Transitions

EXPECTED APPROACH

```
type OrderState =
  | { status: 'pending' }
  | { status: 'shipped'; trackingNumber: string }
  | { status: 'delivered'; deliveredAt: Date }
  | { status: 'cancelled'; reason: string };

function nextState(state: OrderState): OrderState {
  switch (state.status) {
    case 'pending':
      return { status: 'shipped', trackingNumber: 'TRK-0001' };
    case 'shipped':
      return { status: 'delivered', deliveredAt: new Date() };
    case 'delivered':
      return state;
    case 'cancelled':
      return state;
  }
}
```

WHAT TO CHECK

- Each returned object literal includes exactly the fields required by its target discriminant (no missing or extra fields).
- Switch covers all four discriminant values.
- Terminal states (`delivered`, `cancelled`) return the original state object rather than constructing a new one.
- No use of `as OrderState` casts to bypass literal shape checking.

COMMON MISTAKES

- Returning `{ status: 'shipped' }` without the required `trackingNumber` field.
- Forgetting to handle `cancelled` as a distinct terminal case (falling through to a default that changes it).
- Using string literals like `'Shipped'` (wrong case) that don't match the discriminant exactly.
- Casting the return value with `as OrderState` to silence a legitimate missing-field error.

SUGGESTED SCORING

Correct discriminated union transitions	50%
Correct handling of terminal states	30%
No unsafe casts / clean typing	20%

TYPESCRIPT MEDIUM

TS-13 — Sort Items By Key

EXPECTED APPROACH

```
function sortByKey<T, K extends keyof T>(items: T[], key: K): T[] {
  return [...items].sort((a, b) => {
    const aVal = a[key];
    const bVal = b[key];
    if (typeof aVal === 'number' && typeof bVal === 'number') {
      return aVal - bVal;
    }
    return String(aVal).localeCompare(String(bVal));
  });
}
```

WHAT TO CHECK

- Generic constraint `K extends keyof T` ensures `sortByKey(items, 'nonExistentField')` is a compile error.
- Original array is not mutated (uses a copy via spread or `.slice()`).
- Numeric fields are compared numerically, not via string coercion (which would sort '10' before '2').
- Function works generically across different object shapes without `any`.

COMMON MISTAKES

- Using `key: string` instead of `K extends keyof T`, losing compile-time validation of the key name.
- Comparing values directly with `<`/`>` operators on a generic-typed value, which TypeScript rejects for non-primitive-constrained generics.
- Mutating the input array via `.sort()` directly instead of copying first.
- Not handling numeric fields specially, causing incorrect lexicographic sorting of numbers.

SUGGESTED SCORING

Correct keyof generic constraint	40%
Correct non-mutating sort implementation	35%
Correct numeric vs string comparison handling	25%

TYPESCRIPT MEDIUM

TS-14 — HTTP Status Lookup Table

EXPECTED APPROACH

```
type HttpStatusMessages = Record<number, string>;

const statusMessages: HttpStatusMessages = {
  200: 'OK',
  404: 'Not Found',
  500: 'Internal Server Error',
};

function getStatusMessage(messages: HttpStatusMessages, code: number): string {
  return messages[code] ?? 'Unknown Status';
}
```

WHAT TO CHECK

- `HttpStatusMessages` correctly types keys as `number` and values as `string`.
- `getStatusMessage` returns the fallback string for missing codes rather than `undefined`.
- Uses `??` (nullish coalescing) rather than `||`, which would also (incorrectly, though not harmful here) trigger on empty string values.
- The sample `statusMessages` object satisfies the declared type without extra casts.

COMMON MISTAKES

- Using `Record<string, string>` for numeric codes, forcing awkward string-key access.
- Returning `messages[code]` directly, which is `string | undefined` under strict mode and would fail to satisfy a `string` return type.
- Using `||` instead of `??`, which is technically fine here but shows a misunderstanding of the two operators.
- Not marking the lookup table type reusable (inlining the shape ad-hoc at both declaration sites).

SUGGESTED SCORING

Correct Record/index-signature typing	40%
Correct fallback lookup logic	40%
Type-safe handling of possibly-undefined access	20%

TYPESCRIPT MEDIUM

TS-15 — Paginated Data Generator

EXPECTED APPROACH

```
interface Page<T> {
  items: T[];
  nextCursor: string | null;
}

async function* paginate<T>(
  fetchPage: (cursor: string | null) => Promise<Page<T>>
): AsyncGenerator<T, void, unknown> {
  let cursor: string | null = null;
  do {
    const page = await fetchPage(cursor);
    for (const item of page.items) {
      yield item;
    }
    cursor = page.nextCursor;
  } while (cursor !== null);
}
```

WHAT TO CHECK

- Function is declared `async function*` and typed `AsyncGenerator<T, void, unknown>`.
- Loop correctly re-fetches with the updated cursor and terminates when `nextCursor` is `null`.
- `yield`s individual items (`T`), not entire `Page<T>` objects.
- Handles the first call correctly by starting with a `null` cursor.

COMMON MISTAKES

- Declaring the return type as `Promise<T[]>` instead of an async generator, defeating the purpose of streaming.
- Using a `while (true)` loop without a proper termination check on `nextCursor`, risking an infinite loop.
- Yielding the whole `page` object instead of iterating `page.items`.
- Forgetting `await` on `fetchPage(cursor)`, causing a type error (page would be a Promise, not a Page).

SUGGESTED SCORING

Correct async generator typing	40%
Correct pagination loop and termination	40%
Correct per-item yielding	20%

TYPESCRIPT MEDIUM

TS-16 — Typed Counter Reducer

EXPECTED APPROACH

```
interface CounterState {
  count: number;
}

type CounterAction =
  | { type: 'increment'; by: number }
  | { type: 'decrement'; by: number }
  | { type: 'reset' };

function counterReducer(state: CounterState, action: CounterAction): CounterState {
  switch (action.type) {
    case 'increment':
      return { count: state.count + action.by };
    case 'decrement':
      return { count: state.count - action.by };
    case 'reset':
      return { count: 0 };
  }
}
```

WHAT TO CHECK

• `action.by` is accessed only within the `increment`/`decrement` branches where it's guaranteed to exist by the discriminated union.
• `reset` branch does not attempt to read a `by` field (which doesn't exist on that variant).
• Reducer returns a brand-new state object rather than mutating `state.count` directly.
• Switch is exhaustive across all three action types.

COMMON MISTAKES

• Mutating `state.count += action.by; return state;` instead of returning a new object, breaking immutability expectations.
• Trying to access `action.by` on the `reset` branch (compile error) or ignoring the discriminated union and using a generic `action.by` with optional chaining as a workaround.
• Missing a `case` for one of the three action types, causing an implicit-any-like fallthrough (should be a TS error if strict + noImplicitReturns, but is a design flaw regardless).
• Not typing `CounterAction` as a discriminated union at all, using a single interface with optional fields.

SUGGESTED SCORING

Correct discriminated action typing/usage	40%
Correct reducer logic per action	40%
Immutability of returned state	20%

TYPESCRIPT MEDIUM

TS-17 — Service Registry Singleton

EXPECTED APPROACH

```
class ServiceRegistry {
  private static instance: ServiceRegistry;
  private services = new Map<string, unknown>();

  private constructor() {}

  static getInstance(): ServiceRegistry {
    if (!ServiceRegistry.instance) {
      ServiceRegistry.instance = new ServiceRegistry();
    }
    return ServiceRegistry.instance;
  }

  register<T>(key: string, service: T): void {
    this.services.set(key, service);
  }

  resolve<T>(key: string): T {
    const service = this.services.get(key);
    if (service === undefined) {
      throw new Error(`Service not found: ${key}`);
    }
    return service as T;
  }
}
```

WHAT TO CHECK

- Constructor is `private`, preventing `new ServiceRegistry()` from outside the class.
- `getInstance` lazily creates and caches exactly one instance.
- Internal map is typed `Map<string, unknown>` rather than `Map<string, any>`, requiring an explicit cast on retrieval.
- `resolve` throws (rather than returning `undefined as T`) when the key is missing.

COMMON MISTAKES

- Making the constructor public, defeating the singleton guarantee.
- Storing services in a `Map<string, any>`, silently losing type safety on resolve.
- Returning `undefined` cast as `T` when a key is missing instead of throwing, causing confusing downstream null errors.
- Re-creating a new instance on every call to `getInstance` instead of caching it.

SUGGESTED SCORING

Correct singleton enforcement	35%
Correct generic register/resolve typing	40%
Proper error handling on missing key	25%

TYPESCRIPT MEDIUM

TS-18 — Safe JSON Product Parser

EXPECTED APPROACH

```
interface Product {
  id: string;
  name: string;
  price: number;
}

function isProduct(value: unknown): value is Product {
  if (typeof value !== 'object' || value === null) return false;
  const candidate = value as Record<string, unknown>;
  return (
    typeof candidate.id === 'string' &&
    typeof candidate.name === 'string' &&
    typeof candidate.price === 'number'
  );
}

function parseProduct(json: string): Product | null {
  let parsed: unknown;
  try {
    parsed = JSON.parse(json);
  } catch {
    return null;
  }
  return isProduct(parsed) ? parsed : null;
}
```

WHAT TO CHECK

• `JSON.parse`'s result is held in an `unknown`-typed variable, not implicitly `any`.
• `isProduct` checks `typeof value !== 'object' || value === null` before accessing properties, avoiding a crash on primitives/null.
• Each field is checked with `typeof` against its expected primitive type.
• `parseProduct` wraps `JSON.parse` in a try/catch and returns `null` for malformed JSON.

COMMON MISTAKES

• Letting `JSON.parse`'s result flow through as implicit `any`, skipping real validation entirely.
• Not checking `value === null` before treating it as an object (since `typeof null === 'object'`), causing a runtime crash on property access.
• Only checking that fields exist (`'id' in candidate`) without checking their types.
• Forgetting to catch `JSON.parse` exceptions, letting malformed JSON throw uncaught.

SUGGESTED SCORING

Correct unknown-based type guard implementation	45%
Correct handling of parse failures	30%
No unsafe any/implicit trust of external data	25%

TYPESCRIPT MEDIUM

TS-19 — Nested Config Resolver

EXPECTED APPROACH

```
interface AppConfig {
  api?: {
    baseUrl?: string;
    timeout?: number;
  };
  featureFlags?: {
    darkMode?: boolean;
  };
}

function getBaseUrl(config: AppConfig): string {
  return config.api?.baseUrl ?? 'https://api.default.com';
}

function getTimeout(config: AppConfig): number {
  return config.api?.timeout ?? 5000;
}

function isDarkModeEnabled(config: AppConfig): boolean {
  return config.featureFlags?.darkMode ?? false;
}
```

WHAT TO CHECK

- Uses `?.` to traverse `api` and `featureFlags`, which are both typed optional.
- Uses `??` (not `||`) for defaults, correctly preserving an explicit `timeout: 0` if ever set.
- All three functions have concrete, non-optional return types (`string`, `number`, `boolean`).
- No non-null assertions (`!`) used to bypass the optionality instead of proper chaining.

COMMON MISTAKES

- Using `config.api!.baseUrl` with a non-null assertion instead of `?.`, which would crash at runtime if `api` is actually missing.
- Using `||` instead of `??` for the timeout default, which would incorrectly override an explicit `timeout: 0`.
- Writing verbose manual `if (config.api && config.api.baseUrl)` chains instead of optional chaining (not wrong, but misses the intended feature).
- Forgetting a default and returning `string | undefined` instead of a guaranteed `string`.

SUGGESTED SCORING

Correct optional chaining usage	40%
Correct nullish coalescing for defaults	40%
Return types are non-optional as specified	20%

TYPESCRIPT MEDIUM

TS-20 — Tree Search Utility

EXPECTED APPROACH

```
interface TreeNode<T> {
  value: T;
  children: TreeNode<T>[];
}

function findInTree<T>(node: TreeNode<T>, predicate: (value: T) => boolean): TreeNode<T> | null {
  if (predicate(node.value)) return node;
  for (const child of node.children) {
    const found = findInTree(child, predicate);
    if (found) return found;
  }
  return null;
}
```

WHAT TO CHECK

- `TreeNode<T>` is used recursively (children are `TreeNode<T>[]`, not a different shape).
- Search checks the current node's value before descending into children (correct DFS pre-order).
- Recursive call's result is properly checked and propagated up (`if (found) return found`), not discarded.
- Function returns `null` (not `undefined`) when no match exists anywhere, matching the declared return type.

COMMON MISTAKES

- Forgetting to return the recursive call's result, causing the search to always fall through to `null` even on a match.
- Checking children before checking the current node, producing incorrect traversal order.
- Using `any` for `T` or for the tree structure instead of keeping it fully generic.
- Returning `undefined` in some branch while the signature promises `TreeNode<T> | null`.

SUGGESTED SCORING

Correct recursive generic type usage	35%
Correct DFS traversal order	35%
Correct propagation of found results and null	30%

TYPESCRIPT MEDIUM

TS-21 — Typed API Route Builder

EXPECTED APPROACH

```
type ApiVersion = 'v1' | 'v2';
type Resource = 'users' | 'orders' | 'products';

type ApiRoute = `/${ApiVersion}/${Resource}`;

function buildRoute(version: ApiVersion, resource: Resource): ApiRoute {
  return `/${version}/${resource}`;
}

function callApi(route: ApiRoute): string {
  return `Calling ${route}`;
}

// callApi('/v1/usres'); // would be a compile error: not assignable to ApiRoute
```

WHAT TO CHECK

- `ApiRoute` is defined as a template literal type referencing the `ApiVersion` and `Resource` unions (not a plain `string`).
- `buildRoute`'s return type is inferred/declared as `ApiRoute`, and the implementation actually produces a matching string.
- `callApi` accepts only `ApiRoute`, not arbitrary strings.
- Candidate demonstrates (in a comment or explanation) that an invalid literal would fail to compile.

COMMON MISTAKES

- Declaring `ApiRoute` as plain `string`, losing all the compile-time route validation benefit.
- Hardcoding the union of all six literal combinations manually instead of using a template literal type.
- Making `callApi` accept `string` instead of `ApiRoute`, defeating the purpose.
- Building the route with string concatenation whose type doesn't get narrowed back to `ApiRoute` without an explicit type layer (should rely on the template literal type, not a cast).

SUGGESTED SCORING

Correct template literal type definition	45%
Correct buildRoute implementation	30%
Correct restrictive usage in callApi	25%

TYPESCRIPT MEDIUM

TS-22 — Typed Pipe Utility

EXPECTED APPROACH

```
function pipe<A, B, C>(fn1: (a: A) => B, fn2: (b: B) => C): (a: A) => C {
  return (a: A) => fn2(fn1(a));
}

const toUpperCase = (s: string): string => s.toUpperCase();
const exclaim = (s: string): string => `${s}!`;

const shout = pipe(toUpperCase, exclaim);
// shout('hello') === 'HELLO!'

// A, B, C are each necessary because fn1 and fn2 can have different input/output
// types (e.g. pipe(parseInt, isPositive) goes string -> number -> boolean);
// collapsing them to one type parameter would wrongly force fn1 and fn2 to share types.
```

WHAT TO CHECK

- `pipe` is generic over three independent type parameters representing input, intermediate, and output types.
- Returned function correctly threads the input through `fn1` then `fn2` in that order.
- Usage example shows correct type inference without explicit type annotations at the call site.
- Explanation correctly identifies why a single shared type parameter would be too restrictive.

COMMON MISTAKES

- Using a single type parameter `T` for both functions, forcing `fn1` and `fn2` to have matching input/output types unnecessarily.
- Applying the functions in the wrong order (`fn1(fn2(a))` instead of `fn2(fn1(a))`).
- Typing the parameters as `Function` instead of proper generic function signatures, losing type safety entirely.
- Not returning a function at all — eagerly computing a single result instead of a reusable composed function.

SUGGESTED SCORING

Correct 3-parameter generic composition	45%
Correct execution order	30%
Clear demonstration + explanation	25%

TYPESCRIPT MEDIUM

TS-23 — Result Type For Division

EXPECTED APPROACH

```
type Result<T, E = string> = { ok: true; value: T } | { ok: false; error: E };

function divide(a: number, b: number): Result<number> {
  if (b === 0) {
    return { ok: false, error: 'Division by zero' };
  }
  return { ok: true, value: a / b };
}

function handleResult(result: Result<number>): string {
  if (result.ok) {
    return `Result: ${result.value}`;
  }
  return `Error: ${result.error}`;
}
```

WHAT TO CHECK

- `Result` is a discriminated union keyed on a literal boolean `ok` field, with a default error type parameter.
- `divide` never throws for the divide-by-zero case; it returns the `ok: false` variant instead.
- `handleResult` narrows on `result.ok` before accessing `value` or `error`, with no unchecked access.
- Generic default `E = string` is used correctly (callers don't have to specify it for the common case).

COMMON MISTAKES

- Throwing an exception from `divide` on division by zero instead of returning the error variant, defeating the purpose of the Result pattern.
- Accessing `result.value` without first checking `result.ok`, which is a compile error under the discriminated union.
- Making `ok` a plain `boolean` type instead of the literal `true`/`false` per variant, which breaks discrimination.
- Omitting the default type parameter, forcing verbose `Result<number, string>` everywhere.

SUGGESTED SCORING

Correct Result discriminated union design	40%
Correct divide implementation without throwing	35%
Correct narrowing usage in handleResult	25%

TYPESCRIPT MEDIUM

TS-24 — In-Memory Entity Repository

EXPECTED APPROACH

```
interface Entity {
  id: string;
}

interface Repository<T extends Entity> {
  findById(id: string): T | undefined;
  save(entity: T): void;
  delete(id: string): boolean;
  findAll(): T[];
}

class InMemoryRepository<T extends Entity> implements Repository<T> {
  private items = new Map<string, T>();

  findById(id: string): T | undefined {
    return this.items.get(id);
  }

  save(entity: T): void {
    this.items.set(entity.id, entity);
  }

  delete(id: string): boolean {
    return this.items.delete(id);
  }

  findAll(): T[] {
    return Array.from(this.items.values());
  }
}
```

WHAT TO CHECK

- `T extends Entity` constraint is present on both the interface and the class, guaranteeing an `id` field exists.
- `class ... implements Repository<T>` is used, so a missing/mistyped method is caught at compile time.
- `save` correctly uses `entity.id` as the map key (upsert semantics), not a separately-generated key.
- `delete` leverages `Map.delete`'s existing boolean return rather than reimplementing existence-checking logic.

COMMON MISTAKES

- Omitting `implements Repository<T>` on the class, losing the compile-time contract check.
- Forgetting the `T extends Entity` constraint, making `entity.id` inaccessible in `save`.
- Returning `void` from `delete` instead of a `boolean` indicating success.
- Using `any` for the internal map value type instead of `T`.

SUGGESTED SCORING

Correct generic constraint and interface implementation	40%
Correct CRUD method behavior	40%
Idiomatic use of Map semantics	20%

TYPESCRIPT MEDIUM

TS-25 — Typed Publish-Subscribe Channel

EXPECTED APPROACH

```
type Handler<T> = (payload: T) => void;
type Unsubscribe = () => void;

class Channel<T> {
  private handlers: Set<Handler<T>> = new Set();

  subscribe(handler: Handler<T>): Unsubscribe {
    this.handlers.add(handler);
    return () => this.handlers.delete(handler);
  }

  publish(payload: T): void {
    this.handlers.forEach((handler) => handler(payload));
  }
}

interface OrderPlaced {
  orderId: string;
  total: number;
}

const orderChannel = new Channel<OrderPlaced>();
const unsubscribe = orderChannel.subscribe((order) => console.log(order.orderId));
orderChannel.publish({ orderId: 'o1', total: 42 });
unsubscribe();
```

WHAT TO CHECK

- `Unsubscribe` is its own named function type (`() => void`), improving readability over an inline type.
- `subscribe` returns a closure that removes exactly the handler it was given, not all handlers.
- `publish` calls every handler currently in the set with the correctly-typed payload.
- Usage example shows `unsubscribe()` actually being invoked, proving the returned function works.

COMMON MISTAKES

- Using an array with `indexOf`/`splice` in a way that breaks if the same handler function reference is subscribed twice (a `Set` avoids duplicate storage issues cleanly, but candidate should at least handle removal correctly).
- Returning `void` from `subscribe` instead of an unsubscribe function, making cleanup impossible.
- Typing `handler` as `Function` or `(payload: any) => void` instead of `Handler<T>`.
- Forgetting to bind/capture the specific handler in the closure, accidentally removing a different or all handlers.

SUGGESTED SCORING

Correct generic channel typing	35%
Correct subscribe/unsubscribe closure behavior	40%
Correct publish + working demonstration	25%

TYPESCRIPT MEDIUM

TS-26 — Typed Group By Utility

EXPECTED APPROACH

```
function groupBy<T, K extends string | number>(items: T[], keyFn: (item: T) => K): Record<K, T[]> {
  const result = {} as Record<K, T[]>;
  for (const item of items) {
    const key = keyFn(item);
    if (!result[key]) {
      result[key] = [];
    }
    result[key].push(item);
  }
  return result;
}

interface Order {
  id: string;
  status: 'pending' | 'shipped' | 'delivered';
}

const orders: Order[] = [
  { id: '1', status: 'pending' },
  { id: '2', status: 'shipped' },
  { id: '3', status: 'pending' },
];

const grouped = groupBy(orders, (order) => order.status);
```

WHAT TO CHECK

- `K` is constrained to `string | number` so it can be used as a `Record` key.
- Each bucket is lazily initialized before pushing (`if (!result[key])`), avoiding a crash on first insert.
- `keyFn` is a generic parameter, not hardcoded to a specific field name, so the utility is reusable across shapes.
- Demonstration shows correct grouping behavior for at least two distinct keys.

COMMON MISTAKES

- Using `Record<string, T[]>` regardless of `K`, losing the more precise key typing when `K` is a literal union.
- Forgetting to initialize the array before pushing, causing a runtime error on the first item for each new key.
- Hardcoding the grouping field (e.g. always `item.status`) instead of accepting a generic `keyFn`.
- Mutating a shared external object across calls instead of creating a fresh `result` each invocation.

SUGGESTED SCORING

Correct generic key constraint and Record typing	40%
Correct lazy bucket initialization	35%
Working, reusable demonstration	25%

TYPESCRIPT MEDIUM

TS-27 — Typed HTTP Client

EXPECTED APPROACH

```
interface HttpClient {
  get<T>(url: string): Promise<T>;
  post<T, B = unknown>(url: string, body: B): Promise<T>;
}

interface CreateOrderRequest {
  productId: string;
  quantity: number;
}

interface OrderResponse {
  orderId: string;
  status: string;
}

async function createOrder(client: HttpClient, request: CreateOrderRequest): Promise<OrderResponse> {
  return client.post<OrderResponse, CreateOrderRequest>('/orders', request);
}

// B defaults to `unknown` rather than `any` so that callers who omit the body
// type argument still get a type that must be checked/narrowed before use,
// instead of silently disabling type checking on the request body entirely.
```

WHAT TO CHECK

• `client.post` is called with explicit `<OrderResponse, CreateOrderRequest>` type arguments (or types that TS can correctly infer to the same effect).
• `createOrder`'s declared return type matches what `client.post` actually resolves to.
• Explanation correctly distinguishes `unknown`'s safety (forces narrowing) from `any`'s lack of checking.
• No `any` used anywhere in the implementation.

COMMON MISTAKES

• Calling `client.post(url, request)` without type arguments where inference fails to produce `OrderResponse`, resulting in an implicit or unresolved generic.
• Using `any` as the default for `B` instead of `unknown`, which silently disables checking for callers.
• Returning `client.post(...)` without `await`, which still type-checks (Promise<OrderResponse> matches) but shows a misunderstanding of when await is needed — acceptable to note as a discussion point, not a hard type error here.
• Defining a separate ad-hoc response type instead of reusing `OrderResponse`.

SUGGESTED SCORING

Correct generic method invocation with proper type args	45%
Return type correctness	30%
Correct unknown vs any reasoning	25%

TYPESCRIPT MEDIUM

TS-28 — Public User DTO Transform

EXPECTED APPROACH

```
interface DbUser {
  id: string;
  name: string;
  email: string;
  passwordHash: string;
  createdAt: Date;
}

type PublicUser = Omit<DbUser, 'passwordHash'>;

function toPublicUser(user: DbUser): PublicUser {
  const { passwordHash, ...publicUser } = user;
  return publicUser;
}
```

WHAT TO CHECK

• `PublicUser` is derived via `Omit<DbUser, 'passwordHash'>` rather than a manually re-declared interface.
• `toPublicUser` actually removes `passwordHash` at runtime (via destructuring or delete), not just at the type level.
• Return type of `toPublicUser` is `PublicUser`, not `DbUser`, so callers can't accidentally access `passwordHash`.
• No use of `as PublicUser` casts that would bypass actually removing the field at runtime.

COMMON MISTAKES

• Manually rewriting `PublicUser` as a fresh interface listing all fields except `passwordHash`, which drifts out of sync if `DbUser` changes.
• Returning `user as PublicUser` (a cast) without actually stripping `passwordHash` at runtime, so the field is still present on the object despite the type saying otherwise.
• Using `Pick` and manually listing every remaining field instead of `Omit` with just the one excluded field.
• Forgetting to destructure `passwordHash` out, causing the unused-variable pattern to be missing (candidate directly returns `user` typed as `PublicUser`, silently keeping the sensitive field in the actual object).

SUGGESTED SCORING

Correct Omit-based type derivation	40%
Correct runtime field stripping	40%
No unsafe casts / DRY definition	20%

TYPESCRIPT MEDIUM

TS-29 — Typed Debounce Function

EXPECTED APPROACH

```
function debounce<Args extends unknown[]>(
  fn: (...args: Args) => void,
  waitMs: number
): (...args: Args) => void {
  let timeoutId: ReturnType<typeof setTimeout> | undefined;
  return (...args: Args) => {
    if (timeoutId !== undefined) {
      clearTimeout(timeoutId);
    }
    timeoutId = setTimeout(() => fn(...args), waitMs);
  };
}

function search(query: string): void {
  console.log('searching for', query);
}

const debouncedSearch = debounce(search, 300);
debouncedSearch('hello');
```

WHAT TO CHECK

- `Args extends unknown[]` (a tuple/array generic with rest parameters) is used to preserve the wrapped function's exact parameter list.
- `timeoutId` is typed via `ReturnType<typeof setTimeout>` rather than hardcoding a platform-specific timer type.
- Each new call clears any pending timeout before scheduling a new one.
- The returned function's parameters match `fn`'s parameters exactly at the call site (shown via the `search` example).

COMMON MISTAKES

- Using `any[]` for the arguments instead of a generic `Args extends unknown[]`, losing parameter type checking at call sites.
- Typing `timeoutId` as `number`, which is incorrect/unsafe across Node vs browser timer return types.
- Forgetting to clear the previous timeout, causing multiple queued invocations instead of true debouncing.
- Not spreading `args` correctly into the delayed call, or capturing stale arguments incorrectly across closures.

SUGGESTED SCORING

Correct generic parameter preservation	40%
Correct debounce timing/cancellation logic	40%
Correct portable timer typing	20%

TYPESCRIPT MEDIUM

TS-30 — Middleware Pipeline

EXPECTED APPROACH

```
type Middleware<T> = (context: T, next: () => void) => void;

class MiddlewarePipeline<T> {
  private middlewares: Middleware<T>[] = [];

  use(middleware: Middleware<T>): this {
    this.middlewares.push(middleware);
    return this;
  }

  execute(context: T): void {
    let index = 0;
    const next = (): void => {
      const middleware = this.middlewares[index];
      index++;
      if (middleware) {
        middleware(context, next);
      }
    };
    next();
  }
}
```

WHAT TO CHECK

- `use` returns `this`, enabling fluent chaining of multiple `.use()` calls.
- `execute` correctly threads a `next` closure that advances an index and invokes the following middleware.
- Chain naturally halts when a middleware omits calling `next()` (no forced continuation).
- `context` is passed through unchanged to every middleware, typed as `T` (not `any`) throughout.

COMMON MISTAKES

- Iterating middlewares with a plain `for` loop that always calls every middleware regardless of whether `next()` was invoked, breaking the short-circuit contract.
- Returning `void` from `use` instead of `this`, breaking fluent chaining.
- Losing the shared context type by typing it as `any` or `unknown` instead of `T`.
- Off-by-one errors in the index increment causing either an infinite loop or skipping the first/last middleware.

SUGGESTED SCORING

Correct chainable use() typing	30%
Correct next()-driven execution control flow	45%
Correct context typing throughout	25%

REACT MEDIUM

RCT-01 — Build a useToggle Hook

EXPECTED APPROACH

```
function useToggle(initialValue: boolean = false): [boolean, () => void] {
  const [value, setValue] = useState(initialValue);
  const toggle = useCallback(() => setValue((v) => !v), []);
  return [value, toggle];
}

function ToggleDemo() {
  const [isOn, toggle] = useToggle(false);
  return <button onClick={toggle}>{isOn ? "ON" : "OFF"}</button>;
}
```

WHAT TO CHECK

- Hook returns a tuple (array) of [boolean, function], not an object, matching the usage in ToggleDemo.
- Toggle uses a functional state update (setValue(v => !v)) rather than referencing the outer value directly.
- toggle is wrapped in useCallback (or otherwise made referentially stable) so it does not change identity every render.
- Default parameter / typing is correct: initialValue is a boolean, default false.

COMMON MISTAKES

- Writing toggle as () => setValue(!value), which relies on a closure over value and can be stale if called multiple times before re-render.
- Returning an object instead of a tuple, breaking the array-destructuring call site.
- Forgetting useCallback, so every consumer re-renders unnecessarily when passed to memoized children.
- Leaving initialValue and the return type implicitly typed as any under strict mode.

SUGGESTED SCORING

Correct toggle logic and stable identity	45%
Correct hook signature and tuple return	30%
TypeScript typing correctness	25%

REACT MEDIUM

RCT-02 — Fix the Stale Closure Counter

EXPECTED APPROACH

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setCount((c) => c + 1);
    }, 1000);
    return () => clearInterval(id);
  }, []);

  return <p>{count}</p>;
}
```

WHAT TO CHECK

• Identifies that the interval callback closes over count from the first render (stale closure), so it always sets count to 0 + 1.
• Fix uses the functional updater form setCount(c => c + 1) instead of setCount(count + 1).
• Dependency array remains empty ([]) so the interval is created exactly once and not re-created every second.
• Cleanup (clearInterval) is preserved.

COMMON MISTAKES

• Adding count to the dependency array, which 'fixes' the bug by tearing down and recreating the interval every second (works but defeats the purpose and can cause drift).
• Wrapping setCount(count + 1) in useCallback without changing to a functional update — the stale closure bug remains.
• Removing the cleanup function, causing multiple intervals to stack up on re-mount.

SUGGESTED SCORING

Correct root-cause explanation	30%
Correct fix using functional update	50%
Cleanup and empty dependency array preserved	20%

REACT MEDIUM

RCT-03 — Controlled Signup Form Validation

EXPECTED APPROACH

```
interface FormState {
  email: string;
  password: string;
}

interface FormErrors {
  email?: string;
  password?: string;
}

function validate(values: FormState): FormErrors {
  const errors: FormErrors = {};
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(values.email)) {
    errors.email = "Enter a valid email address";
  }
  if (values.password.length < 8) {
    errors.password = "Password must be at least 8 characters";
  }
  return errors;
}

function SignupForm() {
  const [values, setValues] = useState<FormState>({ email: "", password: "" });
  const errors = validate(values);
  const isValid = Object.keys(errors).length === 0;

  function handleChange(field: keyof FormState) {
    return (e: ChangeEvent<HTMLInputElement>) => {
      setValues((prev) => ({ ...prev, [field]: e.target.value }));
    };
  }

  function handleSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    if (isValid) {
      console.log("submitting", values);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={values.email} onChange={handleChange("email")} />
      {errors.email && <span>{errors.email}</span>}
      <input type="password" value={values.password} onChange={handleChange("password")} />
      {errors.password && <span>{errors.password}</span>}
      <button type="submit" disabled={!isValid}>Sign up</button>
    </form>
  );
}
```

REACT MEDIUM

RCT-03 — Controlled Signup Form Validation (continued)

WHAT TO CHECK

- Both inputs use value + onChange (fully controlled), updating state immutably via spread rather than mutation.
- validate() correctly flags a malformed email and a password under 8 characters.
- Submit button's disabled prop is derived from whether errors is empty.
- handleSubmit calls e.preventDefault().

COMMON MISTAKES

- Using defaultValue or leaving inputs uncontrolled, then reading values only on submit.
- Mutating the FormState object directly (values.email = ...) instead of creating a new object.
- Forgetting e.preventDefault(), causing a full page reload on submit.
- Only validating in the submit handler instead of deriving errors from current state on every render, so the disabled state doesn't update as the user types.

SUGGESTED SCORING

Correct validation logic	40%
Correct controlled input wiring	30%
Submit handling and disabled-button logic	20%
Code quality / typing	10%

REACT MEDIUM

RCT-04 — Fix Index as Key Bug

EXPECTED APPROACH

```
function TodoList({ todos }: { todos: { id: string; text: string }[] }) {
  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>
          <input type="checkbox" /> {todo.text}
        </li>
      ))}
    </ul>
  );
}
```

WHAT TO CHECK

- Explanation correctly notes that React uses the key to match old and new elements between renders, and an index key means position (not identity) is used to match, so DOM nodes (and their uncontrolled state like checked) get reused for a different todo after reorder.
- Fix changes the key to todo.id, a stable unique identifier that travels with the item.
- No other unrelated logic is changed.

COMMON MISTAKES

- Believing the bug is in the checkbox itself rather than the key strategy.
- 'Fixing' it with key={index + todo.text}, which is still position-dependent and does not solve reordering.
- Removing the key entirely, which produces a React warning and does not fix the underlying issue.

SUGGESTED SCORING

Correct explanation of index-key + reorder bug	45%
Correct fix using stable id key	45%
No unrelated changes / clean diff	10%

REACT MEDIUM

RCT-05 — Shopping Cart Context and Reducer

EXPECTED APPROACH

```
interface CartItem {
  id: string;
  name: string;
  quantity: number;
}

type CartAction =
  | { type: "ADD_ITEM"; item: { id: string; name: string } }
  | { type: "REMOVE_ITEM"; id: string };

function cartReducer(state: CartItem[], action: CartAction): CartItem[] {
  switch (action.type) {
    case "ADD_ITEM": {
      const existing = state.find((i) => i.id === action.item.id);
      if (existing) {
        return state.map((i) =>
          i.id === action.item.id ? { ...i, quantity: i.quantity + 1 } : i
        );
      }
      return [...state, { ...action.item, quantity: 1 }];
    }
    case "REMOVE_ITEM":
      return state.filter((i) => i.id !== action.id);
    default:
      return state;
  }
}

interface CartContextValue {
  items: CartItem[];
  dispatch: Dispatch<CartAction>;
}

const CartContext = createContext<CartContextValue | undefined>(undefined);

function CartProvider({ children }: { children: ReactNode }) {
  const [items, dispatch] = useReducer(cartReducer, []);
  return (
    <CartContext.Provider value={{ items, dispatch }}>
      {children}
    </CartContext.Provider>
  );
}

function useCart(): CartContextValue {
  const ctx = useContext(CartContext);
  if (!ctx) {
    throw new Error("useCart must be used within CartProvider");
  }
  return ctx;
}
```

REACT MEDIUM

RCT-05 — Shopping Cart Context and Reducer (continued)

```
function AddToCartButton({ id, name }: { id: string; name: string }) {
  const { dispatch } = useCart();
  return (
    <button onClick={() => dispatch({ type: "ADD_ITEM", item: { id, name } })}>
      Add {name}
    </button>
  );
}
```

WHAT TO CHECK

- cartReducer never mutates state; it returns new arrays/objects for every case.
- ADD_ITEM correctly increments quantity for an existing item instead of adding a duplicate row.
- useCart throws when context is undefined, guarding against use outside the provider.
- CartProvider supplies both items and dispatch through context value.

COMMON MISTAKES

- Mutating state with state.push(...) or state[i].quantity++ inside the reducer.
- Forgetting the 'already in cart' branch, so ADD_ITEM always appends a new duplicate entry.
- Returning only items from context (omitting dispatch), forcing consumers to lift dispatch some other way.
- Not guarding useCart, so a missing provider silently returns undefined and crashes later with a confusing error.

SUGGESTED SCORING

Correct, immutable reducer logic	40%
Correct context/provider wiring	30%
useCart guard and hook design	20%
Typing correctness	10%

REACT MEDIUM

RCT-06 — Memoize a Filtered Product List

EXPECTED APPROACH

```
interface Product {
  id: string;
  name: string;
}

const MemoList = memo(function MemoList({ items }: { items: Product[] }) {
  return (
    <ul>
      {items.map((p) => (
        <li key={p.id}>{p.name}</li>
      ))}
    </ul>
  );
});

function ProductList({ products, query }: { products: Product[]; query: string }) {
  const [count, setCount] = useState(0);

  const filtered = useMemo(
    () => products.filter((p) => p.name.includes(query)),
    [products, query]
  );

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Clicks: {count}</button>
      <MemoList items={filtered} />
    </div>
  );
}
```

WHAT TO CHECK

- Explanation correctly identifies that .filter() creates a brand new array every render, so memo's shallow prop comparison always sees a 'changed' items prop.
- Fix wraps the filter call in useMemo with dependency array [products, query].
- MemoList itself is left unchanged (the fix is in the parent, not by removing memo).

COMMON MISTAKES

- Removing memo() from MemoList instead of fixing the reference-stability problem in the parent.
- Adding useMemo but forgetting query or products in the dependency array, causing stale filtered results.
- Using useCallback instead of useMemo (useCallback memoizes functions, not the array value here).

SUGGESTED SCORING

Correct explanation of reference-identity issue	35%
Correct useMemo fix with right dependencies	50%
No unnecessary changes to MemoList	15%

REACT MEDIUM

RCT-07 — Lift State Between Temperature Inputs

EXPECTED APPROACH

```
type Scale = "c" | "f";

function toCelsius(f: number): number {
  return ((f - 32) * 5) / 9;
}

function toFahrenheit(c: number): number {
  return (c * 9) / 5 + 32;
}

function tryConvert(temperature: string, convert: (n: number) => number): string {
  const input = parseFloat(temperature);
  if (Number.isNaN(input)) {
    return "";
  }
  const output = convert(input);
  return output.toFixed(1);
}

function TemperatureInput({
  scale,
  temperature,
  onTemperatureChange,
}: {
  scale: Scale;
  temperature: string;
  onTemperatureChange: (value: string) => void;
}) {
  return (
    <fieldset>
      <legend>{scale === "c" ? "Celsius" : "Fahrenheit"}</legend>
      <input value={temperature} onChange={(e) => onTemperatureChange(e.target.value)} />
    </fieldset>
  );
}

function Calculator() {
  const [temperature, setTemperature] = useState("");
  const [scale, setScale] = useState<Scale>("c");

  const celsius = scale === "f" ? tryConvert(temperature, toCelsius) : temperature;
  const fahrenheit = scale === "c" ? tryConvert(temperature, toFahrenheit) : temperature;

  return (
    <div>
      <TemperatureInput
        scale="c"
        temperature={celsius}
        onTemperatureChange={(value) => {
          setScale("c");
          setTemperature(value);
```

REACT MEDIUM

RCT-07 — Lift State Between Temperature Inputs (continued)

```
        }}
      />
      <TemperatureInput
        scale="f"
        temperature={fahrenheit}
        onTemperatureChange={(value) => {
          setScale("f");
          setTemperature(value);
        }}
      />
    </div>
  );
}
```

WHAT TO CHECK

- State (current temperature string + which scale was last edited) lives in Calculator, not in either TemperatureInput.
- The non-source-of-truth input's displayed value is derived via tryConvert on every render rather than stored separately.
- tryConvert returns an empty string for non-numeric input instead of throwing or rendering NaN.
- Both onTemperatureChange callbacks update both the temperature and which scale is authoritative.

COMMON MISTAKES

- Giving each TemperatureInput its own independent useState, so they immediately fall out of sync.
- Forgetting to track which scale was last edited, causing rounding to compound or the wrong field to be treated as source of truth.
- Letting NaN leak into the rendered value when the user clears the input or types non-numeric text.

SUGGESTED SCORING

Correct lifted-state architecture	45%
Correct conversion / derived-value logic	35%
Handles invalid input gracefully	20%

REACT MEDIUM

RCT-08 — Fix Conditional Rendering Zero Bug

EXPECTED APPROACH

```
function CartSummary({ items }: { items: string[] }) {
  return (
    <div>
      {items.length > 0 ? (
        <ul>
          {items.map((i) => (
            <li key={i}>{i}</li>
          ))}
        </ul>
      ) : (
        <p>Your cart is empty</p>
      )}
    </div>
  );
}
```

WHAT TO CHECK

- Explanation notes that items.length && (...) evaluates to the number 0 (not false) when the array is empty, and React renders numbers (including 0) as text, unlike false/null/undefined which render nothing.
- Fix uses a proper boolean condition (items.length > 0) or a ternary, not a bare && on a number.
- Empty-state message is rendered when there are no items.

COMMON MISTAKES

- Wrapping the whole thing in Boolean(items.length) && ... which works but is a less idiomatic patch than comparing explicitly.
- Fixing the rendering but forgetting to add an actual empty-state message, leaving a blank div.
- Assuming the bug is with the .map() or the key prop instead of the && short-circuit.

SUGGESTED SCORING

Correct explanation of the falsy/zero pitfall	40%
Correct conditional fix	40%
Empty-state UX included	20%

REACT MEDIUM

RCT-09 — Build a Compound Tabs Component

EXPECTED APPROACH

```
interface TabsContextValue {
  activeIndex: number;
  setActiveIndex: (index: number) => void;
}

const TabsContext = createContext<TabsContextValue | undefined>(undefined);

function useTabsContext(): TabsContextValue {
  const ctx = useContext(TabsContext);
  if (!ctx) {
    throw new Error("Tabs components must be used inside <Tabs>");
  }
  return ctx;
}

function TabsBase({
  children,
  defaultIndex = 0,
}: {
  children: ReactNode;
  defaultIndex?: number;
}) {
  const [activeIndex, setActiveIndex] = useState(defaultIndex);
  return (
    <TabsContext.Provider value={{ activeIndex, setActiveIndex }}>
      <div>{children}</div>
    </TabsContext.Provider>
  );
}

function TabList({ children }: { children: ReactNode }) {
  return <div role="tablist">{children}</div>;
}

function Tab({ index, children }: { index: number; children: ReactNode }) {
  const { activeIndex, setActiveIndex } = useTabsContext();
  return (
    <button role="tab" aria-selected={activeIndex === index} onClick={() => setActiveIndex(index)}>
      {children}
    </button>
  );
}

function TabPanel({ index, children }: { index: number; children: ReactNode }) {
  const { activeIndex } = useTabsContext();
  if (activeIndex !== index) return null;
  return <div role="tabpanel">{children}</div>;
}

const Tabs = TabsBase as typeof TabsBase & {
  List: typeof TabList;
```

REACT MEDIUM

RCT-09 — Build a Compound Tabs Component (continued)

```
  Tab: typeof Tab;
  Panel: typeof TabPanel;
};
Tabs.List = TabList;
Tabs.Tab = Tab;
Tabs.Panel = TabPanel;
```

WHAT TO CHECK

- Active tab index lives in context, shared implicitly between Tab and Panel without manual prop threading.
- useTabsContext throws a clear error when used outside a Tabs provider.
- TabPanel renders null (not empty string or undefined) for non-active panels.
- List/Tab/Panel are attached as static properties on Tabs so consumers can write Tabs.Tab, Tabs.Panel.

COMMON MISTAKES

- Prop-drilling activeIndex and setActiveIndex through List manually instead of using context.
- Forgetting to guard useTabsContext, so it silently returns undefined and crashes with a confusing error deep in Tab/Panel.
- Using array index of children instead of an explicit index prop, which breaks if tabs are conditionally rendered or reordered.

SUGGESTED SCORING

Correct context-based state sharing	40%
Correct Tab / Panel active-state logic	35%
Correct compound static-property attachment	25%

REACT MEDIUM

RCT-10 — Build a useDebounce Hook

EXPECTED APPROACH

```
function useDebounce<T>(value: T, delay: number): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timeout = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timeout);
  }, [value, delay]);

  return debounced;
}

function SearchBox() {
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 300);

  useEffect(() => {
    if (debouncedQuery) {
      console.log("searching for", debouncedQuery);
    }
  }, [debouncedQuery]);

  return <input value={query} onChange={(e) => setQuery(e.target.value)} />;
}
```

WHAT TO CHECK

- useDebounce stores its own state initialized to value and updates it via setTimeout.
- The effect's cleanup function calls clearTimeout so rapid changes reset the timer instead of stacking updates.
- Dependency array includes both value and delay.
- SearchBox reacts to debouncedQuery (not query) for the expensive operation.

COMMON MISTAKES

- Forgetting the cleanup/clearTimeout, so every keystroke schedules an extra timeout that still fires later.
- Debouncing by directly mutating a ref and forcing a re-render manually instead of using state.
- Triggering the search effect on query instead of debouncedQuery, defeating the whole purpose of the hook.

SUGGESTED SCORING

Correct debounce timing logic with cleanup	55%
Correct generic typing	20%
Correct consumer wiring in SearchBox	25%

REACT MEDIUM

RCT-11 — Fix Stale Threshold in Resize Listener

EXPECTED APPROACH

```
function WindowWidthLogger() {
  const [width, setWidth] = useState(window.innerWidth);
  const [threshold, setThreshold] = useState(768);

  useEffect(() => {
    function handleResize() {
      if (window.innerWidth < threshold) {
        console.log("below threshold");
      }
      setWidth(window.innerWidth);
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, [threshold]);

  return <p>{width}</p>;
}
```

WHAT TO CHECK

• Explanation correctly identifies that the effect's empty dependency array means handleResize is created once and closes over the threshold from that first render.
• Fix adds threshold to the dependency array so the listener is torn down and re-attached with a fresh closure whenever threshold changes.
• Cleanup (removeEventListener) is preserved so listeners don't accumulate.

COMMON MISTAKES

• Trying to fix it by calling setThreshold inside the effect, which doesn't address the stale-read problem at all.
• Adding threshold to the dependency array but forgetting that this only works because the effect also cleans up the previous listener — removing the cleanup would leak listeners.
• Using a ref for threshold and reading ref.current, which also works but is often introduced without explaining the simpler dependency-array fix first.

SUGGESTED SCORING

Correct root-cause explanation	35%
Correct dependency array fix	45%
Cleanup preserved / no listener leak	20%

REACT MEDIUM

RCT-12 — Fix Uncontrolled Input Warning

EXPECTED APPROACH

```
function EmailField() {
  const [email, setEmail] = useState<string>("");

  return <input value={email} onChange={(e) => setEmail(e.target.value)} />;
}
```

WHAT TO CHECK

- Explanation notes that useState() with no argument starts as undefined, so value={undefined} makes React treat the input as uncontrolled until the first keystroke sets a real string.
- Fix initializes state to an empty string "" instead of leaving it undefined.
- State is typed as string (useState<string>("")) rather than left as an inferred any/undefined.

COMMON MISTAKES

- Switching to defaultValue instead of value, which silences the warning but makes the field uncontrolled and unable to be validated/reset programmatically.
- Using value={email ?? ""} as a band-aid at the JSX level instead of fixing the initial state.
- Leaving the state untyped, so TypeScript infers it as any and strict mode gains nothing.

SUGGESTED SCORING

Correct explanation of controlled/uncontrolled switch	35%
Correct initial state fix	45%
Correct typing	20%

REACT MEDIUM

RCT-13 — Build a Filterable List Component

EXPECTED APPROACH

```
function FilterableList({ items }: { items: string[] }) {
  const [query, setQuery] = useState("");
  const filtered = items.filter((item) =>
    item.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Search..." />
      {filtered.length === 0 ? (
        <p>No results</p>
      ) : (
        <ul>
          {filtered.map((item) => (
            <li key={item}>{item}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

WHAT TO CHECK

- Search input is controlled (value + onChange bound to state).
- Filtering is case-insensitive (both sides lower-cased) and substring-based.
- Keys are derived from the item content/id, not the array index.
- Empty-results state is handled explicitly rather than rendering an empty <ul>.

COMMON MISTAKES

- Using item index as the key even though items are strings that could repeat or reorder after filtering.
- Comparing with === instead of .includes(), only matching exact strings rather than substrings.
- Forgetting to lower-case one side of the comparison, making the filter case-sensitive.

SUGGESTED SCORING

Correct controlled search input	25%
Correct case-insensitive filter logic	35%
Correct keys and empty-state handling	40%

REACT MEDIUM

RCT-14 — Multi Step Wizard with useReducer

EXPECTED APPROACH

```
interface WizardState {
  step: number;
  data: { name: string; email: string };
}

type WizardAction =
  | { type: "NEXT" }
  | { type: "BACK" }
  | { type: "UPDATE_FIELD"; field: keyof WizardState["data"]; value: string };

function wizardReducer(state: WizardState, action: WizardAction): WizardState {
  switch (action.type) {
    case "NEXT":
      return { ...state, step: Math.min(state.step + 1, 3) };
    case "BACK":
      return { ...state, step: Math.max(state.step - 1, 1) };
    case "UPDATE_FIELD":
      return { ...state, data: { ...state.data, [action.field]: action.value } };
    default:
      return state;
  }
}

function Wizard() {
  const [state, dispatch] = useReducer(wizardReducer, { step: 1, data: { name: "", email: "" } });

  return (
    <div>
      <p>Step {state.step} of 3</p>
      {state.step === 1 && (
        <input
          value={state.data.name}
          onChange={(e) => dispatch({ type: "UPDATE_FIELD", field: "name", value: e.target.value })}
        />
      )}
      {state.step === 2 && (
        <input
          value={state.data.email}
          onChange={(e) => dispatch({ type: "UPDATE_FIELD", field: "email", value: e.target.value })}
        />
      )}
      <button onClick={() => dispatch({ type: "BACK" })} disabled={state.step === 1}>Back</button>
      <button onClick={() => dispatch({ type: "NEXT" })} disabled={state.step === 3}>Next</button>
    </div>
  );
}
```

WHAT TO CHECK

- Reducer clamps step within [1, 3] rather than allowing it to run past the boundaries.
- UPDATE_FIELD updates only the targeted field via immutable spread, preserving the other field's value.
- Data entered on step 1 is still present in state if the user goes to step 2 and back.

REACT MEDIUM

RCT-14 — Multi Step Wizard with useReducer (continued)

• Back/Next buttons are disabled at the respective boundary steps.

COMMON MISTAKES

• Using separate useState(step) and useState(data) that can get out of sync instead of one reducer-managed state object.
• Mutating state.data directly (state.data.name = value) inside the reducer.
• Forgetting to clamp NEXT/BACK, letting step go to 0 or 4 and rendering nothing.

SUGGESTED SCORING

Correct reducer logic (immutability + clamping)	45%
Correct per-step rendering	30%
Correct button disabled logic	25%

REACT MEDIUM

RCT-15 — Fix Broken Memoized Button

EXPECTED APPROACH

```
const ExpensiveButton = memo(function ExpensiveButton({
  onClick,
  label,
}: {
  onClick: () => void;
  label: string;
}) {
  console.log("rendering", label);
  return <button onClick={onClick}>{label}</button>;
});

function Toolbar() {
  const [count, setCount] = useState(0);

  const handleSave = useCallback(() => {
    console.log("saved");
  }, []);

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <ExpensiveButton onClick={handleSave} label="Save" />
    </div>
  );
}
```

WHAT TO CHECK

• Explanation notes that handleSave is a new function on every Toolbar render, so memo's shallow prop comparison sees onClick as changed.
• Fix wraps handleSave in useCallback with an empty dependency array (it captures no outer state).
• label prop remains a primitive string, so it doesn't need memoization.

COMMON MISTAKES

• Wrapping the whole Toolbar in memo() instead of stabilizing handleSave — Toolbar has no parent re-rendering it, so this does nothing.
• Moving handleSave outside the component entirely, losing access to any future component-scoped state it might need.
• Using useMemo instead of useCallback for a function value.

SUGGESTED SCORING

Correct explanation of prop-identity issue	35%
Correct useCallback fix	50%
No unrelated/incorrect changes	15%

REACT MEDIUM

RCT-16 — Build a usePrevious Hook

EXPECTED APPROACH

```
function usePrevious<T>(value: T): T | undefined {
  const ref = useRef<T | undefined>(undefined);
  useEffect(() => {
    ref.current = value;
  });
  return ref.current;
}

function PriceTracker({ price }: { price: number }) {
  const previousPrice = usePrevious(price);
  const direction =
    previousPrice === undefined
      ? "initial"
      : price > previousPrice
      ? "up"
      : price < previousPrice
      ? "down"
      : "same";

  return (
    <p>
      {price} ({direction})
    </p>
  );
}
```

WHAT TO CHECK

- usePrevious stores the value in a ref (not state), so updating it doesn't trigger an additional render.
- The ref is updated inside a useEffect with no dependency array (runs after every render) so it reflects the render that just committed, not the one currently in progress.
- usePrevious correctly returns undefined on the very first call.
- PriceTracker correctly branches on previousPrice being undefined before comparing numerically.

COMMON MISTAKES

- Updating ref.current directly during render (not inside an effect), which would make usePrevious return the *current* value instead of the previous one.
- Using useState instead of useRef, causing an unnecessary extra render every time the tracked value changes.
- Forgetting to special-case previousPrice === undefined, causing a wrong 'up'/'down' on the very first render.

SUGGESTED SCORING

Correct ref-based implementation with effect timing	55%
Correct generic typing	20%
Correct usage / direction logic in PriceTracker	25%

REACT MEDIUM

RCT-17 — Mouse Tracker Render Prop

EXPECTED APPROACH

```
import React, { useState } from "react";

interface Point {
  x: number;
  y: number;
}

function MouseTracker({ render }: { render: (point: Point) => React.ReactNode }) {
  const [point, setPoint] = useState<Point>({ x: 0, y: 0 });

  function handleMouseMove(e: React.MouseEvent<HTMLDivElement>) {
    setPoint({ x: e.clientX, y: e.clientY });
  }

  return (
    <div onMouseMove={handleMouseMove} style={{ height: 200 }}>
      {render(point)}
    </div>
  );
}

function MouseTrackerExample() {
  return (
    <MouseTracker
      render={(point) => (
        <p>
          Mouse position: {point.x}, {point.y}
        </p>
      )}
    />
  );
}
```

WHAT TO CHECK

• MouseTracker holds { x, y } in its own state, updated from a mousemove handler.
• The render prop is called with the current point and its return value is rendered directly (not stored separately).
• handleMouseMove reads clientX/clientY from the event, correctly typed as a React.MouseEvent<HTMLDivElement>.
• Example usage demonstrates the separation between tracking logic (MouseTracker) and presentation (the render function).

COMMON MISTAKES

• Hardcoding what gets rendered inside MouseTracker instead of delegating to the render prop, defeating the reusability goal.
• Attaching the mousemove listener to window/document instead of the wrapping div without adjusting coordinates, or forgetting cleanup if window is used.
• Leaving the event parameter implicitly typed as any instead of React.MouseEvent<HTMLDivElement>.

SUGGESTED SCORING

Correct render-prop pattern	40%
Correct mouse-position state/handler	35%
Correct typing	25%

REACT MEDIUM

RCT-18 — Fix a Fetch Race Condition

EXPECTED APPROACH

```
interface User {
  id: string;
  name: string;
}

declare function fetchUser(id: string): Promise<User>;

function UserProfile({ userId }: { userId: string }) {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    let ignore = false;
    fetchUser(userId).then((data) => {
      if (!ignore) {
        setUser(data);
      }
    });
    return () => {
      ignore = true;
    };
  }, [userId]);

  return <p>{user ? user.name : "Loading..."}</p>;
}
```

WHAT TO CHECK

- Explanation correctly describes the race: effect for userId A fires a slow request, userId changes to B and fires a fast request that resolves first, then A's response arrives later and overwrites B's data.
- Fix introduces a per-effect-run flag (e.g. ignore) that is set to true in the cleanup function.
- setUser is only called when the flag indicates this effect run is still the latest one.
- Dependency array remains [userId].

COMMON MISTAKES

- Trying to fix it by adding a loading boolean alone, which doesn't prevent the stale response from still overwriting state.
- Using a single module-level or ref variable shared across all instances instead of a per-effect-closure flag, breaking if multiple UserProfile instances exist.
- Forgetting to actually check the flag before calling setUser, only setting it in cleanup without reading it.

SUGGESTED SCORING

Correct explanation of the race condition	30%
Correct ignore-flag / cleanup fix	55%
Dependency array and effect structure otherwise intact	15%

REACT MEDIUM

RCT-19 — Fix Mutated Checkbox Group State

EXPECTED APPROACH

```
function PreferencesForm() {
  const options = ["email", "sms", "push"];
  const [selected, setSelected] = useState<string[]>([]);

  function handleChange(option: string) {
    setSelected((prev) =>
      prev.includes(option) ? prev.filter((o) => o !== option) : [...prev, option]
    );
  }

  return (
    <div>
      {options.map((option) => (
        <label key={option}>
          <input
            type="checkbox"
            checked={selected.includes(option)}
            onChange={() => handleChange(option)}
          />
          {option}
        </label>
      ))}
    </div>
  );
}
```

WHAT TO CHECK

• Explanation identifies both the mutation (selected.push mutates the array in place) and the missing toggle-off branch (it only ever adds, never removes).
• Fix uses a functional state update that branches on whether the option is already selected.
• No direct mutation of the selected array remains anywhere in the fix.
• checked={selected.includes(option)} logic is left intact / still correctly derives checked state.

COMMON MISTAKES

• Fixing only the mutation (using [...selected, option]) but still never handling the uncheck case.
• Fixing only the toggle logic but keeping selected.push, still mutating the same array reference React compares against.
• Comparing against a stale selected captured in the outer closure instead of using the functional updater form.

SUGGESTED SCORING

Correct explanation of both bugs	30%
Correct immutable toggle logic	50%
No remaining mutation	20%

REACT MEDIUM

RCT-20 — Fix Mutated Shopping List State

EXPECTED APPROACH

```
function ShoppingList() {
  const [items, setItems] = useState<string[]>(["Milk"]);
  const [text, setText] = useState("");

  function addItem() {
    setItems((prev) => [...prev, text]);
    setText("");
  }

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={addItem}>Add</button>
      <ul>
        {items.map((item, i) => (
          <li key={i}>{item}</li>
        ))}
      </ul>
    </div>
  );
}
```

WHAT TO CHECK

- Explanation notes that .push() mutates the existing array in place, so the reference passed to setItems is identical (===) to the previous state, and React's bailout on unchanged state skips the re-render.
- Fix creates a new array (e.g. via spread) instead of mutating items.
- text is still cleared after adding.

COMMON MISTAKES

- Calling setItems([...items, text]) but leaving the original items.push(text) line in place, still mutating before the copy is made (harmless here but a red flag habit).
- Forcing a re-render with a key change or forceUpdate-style hack instead of fixing the state update itself.
- Forgetting to clear text after adding, leaving stale text in the input.

SUGGESTED SCORING

Correct explanation of mutation / reference equality	35%
Correct immutable update fix	50%
Input cleared after add	15%

REACT MEDIUM

RCT-21 — Theme Toggle with Context

EXPECTED APPROACH

```
type Theme = "light" | "dark";

interface ThemeContextValue {
  theme: Theme;
  toggleTheme: () => void;
}

const ThemeContext = createContext<ThemeContextValue | undefined>(undefined);

function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<Theme>("light");

  const toggleTheme = useCallback(() => {
    setTheme((t) => (t === "light" ? "dark" : "light"));
  }, []);

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

function useTheme(): ThemeContextValue {
  const ctx = useContext(ThemeContext);
  if (!ctx) {
    throw new Error("useTheme must be used within a ThemeProvider");
  }
  return ctx;
}

function ThemeToggleButton() {
  const { theme, toggleTheme } = useTheme();
  return <button onClick={toggleTheme}>Current: {theme}</button>;
}
```

WHAT TO CHECK

- Theme state lives in ThemeProvider and is exposed via context, not global variables or prop drilling.
- toggleTheme correctly flips between 'light' and 'dark' using a functional update.
- useTheme guards against missing provider with a thrown error.
- ThemeToggleButton consumes context purely through useTheme, with no direct useContext(ThemeContext) call.

COMMON MISTAKES

- Passing theme and setTheme (the raw setter) through context instead of a semantic toggleTheme function, leaking implementation details to consumers.
- Forgetting the undefined guard in useTheme, causing a silent crash with a confusing error far from the real cause.
- Re-creating the context value object without useMemo/useCallback in a way that's fine here but would cause unnecessary re-renders in a larger provider (worth a passing mention).

SUGGESTED SCORING

REACT MEDIUM

RCT-21 — Theme Toggle with Context (continued)

Correct context + provider design	40%
Correct toggle logic	30%
Correct useTheme guard and hook design	30%

REACT MEDIUM

RCT-22 — Memoize an Expensive Total

EXPECTED APPROACH

```
interface Order {
  id: string;
  amount: number;
}

function AnalyticsPanel({ orders }: { orders: Order[] }) {
  const [filter, setFilter] = useState("");

  const total = useMemo(() => {
    console.log("computing total...");
    return orders.reduce((sum, o) => sum + o.amount, 0);
  }, [orders]);

  return (
    <div>
      <input value={filter} onChange={(e) => setFilter(e.target.value)} />
      <p>Total: {total}</p>
    </div>
  );
}
```

WHAT TO CHECK

• Explanation notes that computeTotal is called unconditionally on every render of AnalyticsPanel, and typing in the filter input causes a re-render regardless of orders.
• Fix wraps the computation in useMemo with dependency array [orders].
• The console.log stays inside the memoized computation so it only logs when orders changes (a good way to verify the fix on paper).

COMMON MISTAKES

• Memoizing with an empty dependency array [], which stops the total from ever updating even when orders genuinely changes.
• Debouncing the filter input instead of addressing the actual expensive computation, treating the symptom rather than the cause.
• Moving computeTotal outside the component (module scope) without memoization, which avoids recreating the function but does not avoid re-running it every render.

SUGGESTED SCORING

Correct explanation of unconditional recomputation	30%
Correct useMemo fix with right dependency	55%
No regression in correctness of the total	15%

REACT MEDIUM

RCT-23 — Build a useLocalStorage Hook

EXPECTED APPROACH

```
function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T) => void] {
  const [value, setValue] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? (JSON.parse(item) as T) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    window.localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

function NameField() {
  const [name, setName] = useLocalStorage<string>("name", "");
  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

WHAT TO CHECK

- Initial state is computed lazily (a function passed to useState) rather than reading localStorage on every render.
- A try/catch (or equivalent guard) prevents a JSON.parse failure from crashing the component.
- An effect persists value to localStorage whenever key or value changes.
- Returned tuple mirrors useState's shape so it's a drop-in replacement.

COMMON MISTAKES

- Reading localStorage.getItem directly as the useState initial argument (not wrapped in a function), causing it to run on every render instead of once.
- Forgetting JSON.stringify/JSON.parse, storing/reading raw strings that break for non-string T.
- Omitting the try/catch, so malformed existing localStorage data throws and crashes the component on mount.

SUGGESTED SCORING

Correct lazy-init + persistence logic	50%
Error handling for malformed storage data	25%
Correct generic typing / API shape	25%

REACT MEDIUM

RCT-24 — Loading Error Success State Machine

EXPECTED APPROACH

```
interface Post {
  id: string;
  title: string;
}

declare function fetchPosts(): Promise<Post[]>;

type RequestState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };

function PostsView() {
  const [state, setState] = useState<RequestState<Post[]>>({ status: "idle" });

  useEffect(() => {
    setState({ status: "loading" });
    fetchPosts()
      .then((data) => setState({ status: "success", data }))
      .catch((err: Error) => setState({ status: "error", error: err.message }));
  }, []);

  if (state.status === "idle" || state.status === "loading") {
    return <p>Loading...</p>;
  }
  if (state.status === "error") {
    return <p>Error: {state.error}</p>;
  }
  return (
    <ul>
      {state.data.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

WHAT TO CHECK

• A single state variable typed as the discriminated union drives all rendering, with no separate boolean flags.
• Each render branch narrows on state.status before accessing state.error or state.data, relying on TypeScript's discriminated-union narrowing.
• The effect transitions status: idle -> loading -> success/error correctly.
• success state renders using state.data, error state using state.error — never mixed up.

COMMON MISTAKES

• Keeping a hybrid approach (union state plus a leftover isLoading boolean), reintroducing the original illegal-state problem.
• Accessing state.data without first narrowing status to 'success', which TypeScript strict mode should reject.
• Forgetting to set status to 'loading' before the fetch starts, so the idle UI flashes briefly first.

REACT MEDIUM

RCT-24 — Loading Error Success State Machine (continued)

SUGGESTED SCORING

Correct discriminated-union modeling	40%
Correct state transitions in the effect	30%
Correct narrowed rendering per status	30%

REACT MEDIUM

RCT-25 — Compose a Card with Children

EXPECTED APPROACH

```
function Card({ children }: { children: ReactNode }) {
  return <div className="card">{children}</div>;
}

function CardHeader({ children }: { children: ReactNode }) {
  return <div className="card-header">{children}</div>;
}

function CardBody({ children }: { children: ReactNode }) {
  return <div className="card-body">{children}</div>;
}

function ProductCard({ title, description }: { title: string; description: string }) {
  return (
    <Card>
      <CardHeader>{title}</CardHeader>
      <CardBody>{description}</CardBody>
    </Card>
  );
}
```

WHAT TO CHECK

• ProductCard uses Card/CardHeader/CardBody as-is via composition (children), without adding new props to those base components.
• title lands inside CardHeader and description inside CardBody, in the correct slots.
• ProductCard's own props (title, description) are correctly typed as strings.

COMMON MISTAKES

• Adding a title/description prop directly onto Card or CardHeader instead of composing via children, coupling the generic layout components to product-specific data.
• Skipping CardHeader/CardBody and putting both title and description as direct children of Card, losing the intended header/body separation.
• Duplicating the layout div/className structure inside ProductCard instead of reusing the existing components.

SUGGESTED SCORING

Correct composition using existing components	55%
Correct slot placement (header vs body)	30%
Correct typing	15%

REACT MEDIUM

RCT-26 — Fix Missing Effect Dependency

EXPECTED APPROACH

```
function Greeting({ name }: { name: string }) {
  const [greeting, setGreeting] = useState("");

  useEffect(() => {
    setGreeting(`Hello, ${name}!`);
  }, [name]);

  return <p>{greeting}</p>;
}
```

WHAT TO CHECK

- Explanation identifies the empty dependency array as the cause: the effect runs exactly once on mount and never again when name changes.
- Fix adds name to the dependency array so the effect re-runs and calls setGreeting with the latest name.
- Bonus/ideal alternative noted or accepted: deriving greeting directly as a computed value (const greeting = `Hello, ${name}!`) instead of storing it in state at all, since it doesn't depend on anything but the prop.

COMMON MISTAKES

- Adding an eslint-disable comment to silence the missing-dependency warning instead of actually fixing it.
- Adding name to the dependency array but leaving unrelated derived state in useState when it could be computed directly during render.
- Believing the bug is in the JSX/rendering rather than the effect's dependency array.

SUGGESTED SCORING

Correct root-cause explanation	35%
Correct dependency array fix	45%
Recognizes the simpler derive-without-state alternative (or fix is otherwise clean)	20%

REACT MEDIUM

RCT-27 — Build a useOnClickOutside Hook

EXPECTED APPROACH

```
function useOnClickOutside<T extends HTMLElement>(
  ref: RefObject<T | null>,
  handler: () => void
): void {
  useEffect(() => {
    function listener(event: MouseEvent) {
      const el = ref.current;
      if (!el || el.contains(event.target as Node)) {
        return;
      }
      handler();
    }
    document.addEventListener("mousedown", listener);
    return () => document.removeEventListener("mousedown", listener);
  }, [ref, handler]);
}

function Dropdown() {
  const [open, setOpen] = useState(false);
  const ref = useRef<HTMLDivElement>(null);
  useOnClickOutside(ref, () => setOpen(false));

  return (
    <div ref={ref}>
      <button onClick={() => setOpen((o) => !o)}>Menu</button>
      {open && (
        <ul>
          <li>Option 1</li>
        </ul>
      )}
    </div>
  );
}
```

WHAT TO CHECK

- Listener is attached to document (mousedown or click), not to the referenced element itself.
- Uses el.contains(event.target) to distinguish inside vs outside clicks, guarding against a null ref.current.
- Effect cleans up by removing the listener.
- Dropdown correctly attaches the ref to its outer wrapping element so the whole menu (button + list) counts as 'inside'.

COMMON MISTAKES

- Attaching the listener to the ref element itself (which would never fire for outside clicks), instead of document.
- Forgetting the ref.current null check, causing a crash before the element has mounted or after it has unmounted.
- Omitting the cleanup function, leaking one listener per mount/dropdown instance.

SUGGESTED SCORING

Correct outside-click detection logic	45%
Correct null-safety and cleanup	30%
Correct usage/typing in Dropdown	25%

REACT MEDIUM

RCT-28 — Fix Key Bug in Contact Form

EXPECTED APPROACH

```
function ContactForm({ contacts }: { contacts: { id: string; name: string }[] }) {
  const [localContacts, setLocalContacts] = useState(contacts);

  function removeContact(id: string) {
    setLocalContacts((prev) => prev.filter((c) => c.id !== id));
  }

  return (
    <div>
      {localContacts.map((contact) => (
        <div key={contact.id}>
          <input defaultValue={contact.name} />
          <button onClick={() => removeContact(contact.id)}>Remove</button>
        </div>
      ))}
    </div>
  );
}
```

WHAT TO CHECK

• Explanation notes that with an index key, removing an item shifts every subsequent item's index, so React matches the existing DOM input (with its live, possibly-edited uncontrolled value) to a different contact than before, and defaultValue is not reapplied to already-mounted DOM nodes.
• Fix changes the key from index to contact.id, a value that doesn't shift when items are removed.
• No change needed to defaultValue itself — using a controlled input would also fix it but isn't required if the key is fixed.

COMMON MISTAKES

• Switching to a controlled input as the only fix while leaving key={index}, which happens to mask the symptom in this exact case but doesn't address the root cause and can misbehave in other scenarios (e.g. reordering).
• Assuming the bug is in removeContact's filter logic rather than the key/reconciliation behavior.
• Using key={contact.name} instead of key={contact.id}, which breaks again if two contacts share a name.

SUGGESTED SCORING

Correct explanation of uncontrolled-input + index-key interaction	45%
Correct fix using stable id key	40%
No unrelated changes	15%

REACT MEDIUM

RCT-29 — Lift State for Search Filtering

EXPECTED APPROACH

```
interface UserItem {
  id: string;
  name: string;
}

function SearchBox({ value, onChange }: { value: string; onChange: (value: string) => void }) {
  return <input value={value} onChange={(e) => onChange(e.target.value)} placeholder="Search users..." />;
}

function UserList({ users, query }: { users: UserItem[]; query: string }) {
  const filtered = users.filter((u) => u.name.toLowerCase().includes(query.toLowerCase()));
  return (
    <ul>
      {filtered.map((u) => (
        <li key={u.id}>{u.name}</li>
      ))}
    </ul>
  );
}

function SearchableUserList({ users }: { users: UserItem[] }) {
  const [query, setQuery] = useState("");

  return (
    <div>
      <SearchBox value={query} onChange={setQuery} />
      <UserList users={users} query={query} />
    </div>
  );
}
```

WHAT TO CHECK

- query state lives only in SearchableUserList; SearchBox and UserList remain unmodified and stateless.
- SearchBox receives value and onChange (setQuery passed directly works since its signature matches).
- UserList receives the same query value so its filtering stays in sync with what's typed.
- Neither sibling talks to the other directly — all communication flows through the lifted parent state.

COMMON MISTAKES

- Giving SearchBox its own internal useState for the input value, causing it to fall out of sync with UserList's filter.
- Modifying UserList or SearchBox's props/behavior instead of composing them as given.
- Passing an inline wrapper like onChange={(v) => setQuery(v)} unnecessarily where onChange={setQuery} would do (not wrong, but worth noting as simplification).

SUGGESTED SCORING

Correct lifted-state placement	50%
Correct prop wiring to both siblings	35%
No modification of the reusable child components	15%

REACT MEDIUM

RCT-30 — Fix Memo Object Identity Bug

EXPECTED APPROACH

```
const UserCard = memo(function UserCard({ user }: { user: { name: string } }) {
  console.log("rendering", user.name);
  return <p>{user.name}</p>;
});

const staticUser = { name: "Ada" };

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>{count}</button>
      <UserCard user={staticUser} />
    </div>
  );
}
```

WHAT TO CHECK

• Explanation notes that the inline object literal { name: "Ada" } is a brand-new object on every App render, so memo's shallow comparison (===) always sees a changed user prop.
• Fix hoists the object so the same reference is reused across renders — either moved outside the component (as here, since it's static) or wrapped in useMemo if it depended on props/state.
• UserCard itself is left unchanged; the fix lives entirely in how App constructs the prop.

COMMON MISTAKES

• Adding a custom comparison function to memo() instead of fixing the underlying reference-identity problem at the source.
• Wrapping user in useMemo(() => ({ name: "Ada" }), []) inside App — functionally fine, but candidates should recognize that hoisting a truly static object outside the component entirely is simpler and avoids the hook.
• Removing memo() altogether instead of fixing the prop identity.

SUGGESTED SCORING

Correct explanation of object-identity issue	35%
Correct fix (hoisted or memoized stable reference)	50%
No unnecessary removal of memo	15%

CSS MEDIUM

CSS-01 — Vertically Centered Login Card

EXPECTED APPROACH

```
<div class="flex h-screen items-center justify-center bg-gray-100">
  <div class="w-96 bg-white p-8 rounded-lg shadow-md">
    <h2 class="text-xl font-bold">Sign in</h2>
    <p>Email and password fields go here.</p>
  </div>
</div>
```

WHAT TO CHECK

- items-center is added for cross-axis (vertical) centering.
- justify-center is added for main-axis (horizontal) centering.
- h-screen is retained so the flex container actually has full viewport height to center within.
- No absolute positioning or margin hacks introduced.

COMMON MISTAKES

- Adding only justify-center, assuming it centers on both axes.
- Confusing items- (cross-axis) with justify- (main-axis) in a row-direction flex container.
- Using text-center on the card, which only centers inline text, not the block itself.
- Removing h-screen, which removes the vertical space needed for centering to be visible.

SUGGESTED SCORING

Correct items-center for vertical centering	40%
Correct justify-center for horizontal centering	40%
Explanation of axis distinction	20%

CSS MEDIUM

CSS-02 — Responsive Photo Gallery Grid

EXPECTED APPROACH

```
<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
  <img src="a.jpg" class="w-full"/>
  <img src="b.jpg" class="w-full"/>
  <img src="c.jpg" class="w-full"/>
  <img src="d.jpg" class="w-full"/>
</div>
```

WHAT TO CHECK

- Base (unprefixed) class is grid-cols-1 for mobile-first default.
- sm: breakpoint sets grid-cols-2.
- lg: breakpoint sets grid-cols-4.
- gap-4 (or similar) is present at every size, and grid (not flex) is used.

COMMON MISTAKES

- Starting from grid-cols-4 and never overriding it, keeping mobile broken.
- Using md: instead of sm: for the 2-column tablet step (still valid but skips the intended breakpoint).
- Forgetting the unprefixed base class, leaving mobile at whatever grid-cols-4 was doing.
- Dropping gap-4, causing images to touch edge-to-edge.

SUGGESTED SCORING

Correct base + sm + lg column classes	60%
Gap retained at all sizes	15%
Mobile-first ordering understanding	25%

CSS MEDIUM

CSS-03 — Card Component with Variant Modifiers

EXPECTED APPROACH

```
<div class="p-6 rounded-lg shadow bg-white">
  <h3 class="font-bold">Standard Plan</h3>
  <p>Plan description text.</p>
</div>

<div class="p-6 rounded-lg shadow-lg bg-white border-2 border-blue-500">
  <h3 class="font-bold">Featured Plan</h3>
  <p>Plan description text.</p>
</div>

<div class="p-2 rounded-lg shadow bg-white">
  <h3 class="font-bold text-sm">Compact Plan</h3>
  <p class="text-sm">Plan description text.</p>
</div>
```

WHAT TO CHECK

- Base classes (rounded-lg, bg-white, shadow family) stay consistent across all three cards.
- Featured variant visibly adds a border and a stronger shadow (shadow-lg) than the base.
- Compact variant clearly reduces padding (e.g. p-2 instead of p-6) and text size.
- No single element carries two contradictory sizing classes (e.g. both p-6 and p-2).

COMMON MISTAKES

- Applying both shadow and shadow-lg on the same element, relying on unpredictable class order instead of picking one.
- Forgetting rounded-lg on the featured or compact variant, breaking visual consistency of the family.
- Using inline style attributes instead of utility classes for the variant differences.
- Making the compact variant smaller only in padding but not in text size, defeating the density goal.

SUGGESTED SCORING

Shared base classes stay consistent	40%
Featured variant correctness	30%
Compact variant correctness	30%

CSS MEDIUM

CSS-04 — Button Variant System

EXPECTED APPROACH

```
<button class="px-4 py-2 rounded font-medium bg-blue-600 text-white hover:bg-blue-700">Primary</button>
<button class="px-4 py-2 rounded font-medium border border-gray-400 text-gray-700 hover:bg-gray-100">Secondary
<button class="px-4 py-2 rounded font-medium bg-red-600 text-white hover:bg-red-700">Danger</button>
```

WHAT TO CHECK

- px-4 py-2 rounded font-medium (or equivalent) identical across all three buttons.
- Each variant has a distinct, correct background/border color for its purpose.
- hover: prefix is used, not a permanently-applied darker color.
- Text color contrasts correctly (white on solid colors, dark on the light secondary button).

COMMON MISTAKES

- Forgetting the hover: prefix and applying bg-blue-700 directly, making the button permanently dark.
- Giving each button different padding, breaking the shared base look.
- Leaving Secondary without a border, making it look like plain unstyled text.
- Using text-white on the Secondary button, making the label invisible against its light background.

SUGGESTED SCORING

Shared base classes across variants	30%
Correct variant colors	40%
Correct hover states	30%

CSS MEDIUM

CSS-05 — Sticky Footer Layout

EXPECTED APPROACH

```
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  margin: 0;
}

main {
  flex: 1;
}
```

WHAT TO CHECK

- body (or a wrapper) uses display: flex with flex-direction: column.
- min-height: 100vh is used (not height: 100vh, which would clip long content).
- main is given flex: 1 (or flex-grow: 1) so it expands to push the footer down.
- No position: fixed or position: absolute used on the footer.

COMMON MISTAKES

- Using position: fixed on the footer, which overlaps short content instead of sitting below it.
- Using height: 100vh instead of min-height: 100vh, clipping content taller than the viewport.
- Forgetting flex-direction: column, leaving header/main/footer side by side in a row.
- Forgetting flex: 1 on main, so the footer does not get pushed to the bottom on short pages.

SUGGESTED SCORING

flex column + min-height:100vh on wrapper	40%
flex:1 on main content	40%
Avoids fixed/absolute positioning hacks	20%

CSS MEDIUM

CSS-06 — Holy Grail Layout with CSS Grid

EXPECTED APPROACH

```
.layout {
  display: grid;
  grid-template-columns: 200px 1fr 150px;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header header"
    "nav main aside"
    "footer footer footer";
  min-height: 100vh;
}
.header { grid-area: header; }
.nav    { grid-area: nav; }
.main   { grid-area: main; }
.aside  { grid-area: aside; }
.footer { grid-area: footer; }
```

WHAT TO CHECK

- grid-template-areas rows each contain exactly 3 tokens, matching the 3 defined columns.
- grid-template-columns matches the layout: 200px / 1fr / 150px in the nav/main/aside order.
- header and footer area names repeat across all 3 column positions in their rows to span full width.
- Every child element has a grid-area declaration matching a name used in the template.

COMMON MISTAKES

- Mismatched token counts per row versus the number of defined columns, which makes the whole grid-template-areas rule invalid.
- Forgetting to repeat 'header'/'footer' across all three columns, leaving them confined to one cell.
- Omitting grid-area on one of the child elements, so it falls back to normal auto-placement.
- Typo mismatch between a name used in grid-template-areas and the grid-area value on the element.

SUGGESTED SCORING

Valid grid-template-areas syntax with consistent columns	40%
Correct column sizing (200px / 1fr / 150px)	30%
Correct grid-area assignment on every child	30%

CSS MEDIUM

CSS-07 — Flex Child Text Overflow Bug

EXPECTED APPROACH

```
<div class="flex items-center gap-2 w-64 border p-2">
  <svg class="w-5 h-5 shrink-0"></svg>
  <span class="min-w-0 truncate">this-is-a-very-long-filename-that-should-truncate.pdf</span>
</div>
```

WHAT TO CHECK

- Candidate identifies that flex items default to min-width: auto, which prevents shrinking below their content's intrinsic width.
- min-w-0 is added to the span (the shrinking child), not the parent.
- truncate class is kept on the span.
- shrink-0 stays on the icon so only the text shrinks, not the icon.

COMMON MISTAKES

- Adding overflow-hidden to the parent container instead of min-w-0 on the child, which does not fix the underlying flex sizing issue.
- Removing shrink-0 from the icon, letting the icon itself shrink or distort.
- Wrapping the text with break-all instead of truncating it to a single line.
- Not recognizing the flex min-width:auto default as the root cause.

SUGGESTED SCORING

Correct diagnosis of the min-width:auto flex default	40%
Correct fix (min-w-0 on the shrinking child)	40%
Rest of the row layout (icon, gap, width) preserved	20%

CSS MEDIUM

CSS-08 — Responsive Bootstrap Navbar

EXPECTED APPROACH

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <div class="container">
    <a class="navbar-brand" href="#">Brand</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navMenu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">About</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>
```

WHAT TO CHECK

- navbar-expand-lg used to set the breakpoint at which the menu switches from collapsed to inline.
- data-bs-target on the toggler button matches the id on the collapse div.
- Links are structured as li.nav-item > a.nav-link inside ul.navbar-nav.
- container wraps the navbar content for centering and side padding.

COMMON MISTAKES

- Mismatched data-bs-target and collapse div id, so the toggler does nothing.
- Forgetting the navbar-toggler-icon span inside the toggler button.
- Using plain div elements instead of ul/li for the nav-item structure.
- Omitting navbar-expand-{breakpoint}, so the menu is always collapsed or never collapses.

SUGGESTED SCORING

Correct navbar/toggler/collapse structure	50%
Correct expand breakpoint class	20%
Correct nav-item/nav-link markup	30%

CSS MEDIUM

CSS-09 — Status Badge Positioned on an Avatar

EXPECTED APPROACH

```
<div class="relative w-16 h-16 rounded-full bg-gray-300">
  <span class="absolute bottom-0 right-0 w-4 h-4 bg-green-500 rounded-full border-2 border-white"></span>
</div>
```

WHAT TO CHECK

- relative added on the avatar parent.
- absolute added on the badge.
- bottom-0 right-0 used to pin the badge to the bottom-right corner.
- border-2 border-white added for contrast against the avatar.

COMMON MISTAKES

- Forgetting relative on the parent, so the badge positions relative to the nearest positioned ancestor or the viewport instead.
- Using top-0 left-0 instead of bottom-0 right-0, placing the badge in the wrong corner.
- Forgetting rounded-full on the badge itself, leaving it square.
- Using margin-based offsets instead of absolute positioning, which breaks at different avatar sizes.

SUGGESTED SCORING

relative/absolute pairing on parent/child	40%
Correct corner offset classes	35%
White border ring for contrast	25%

CSS MEDIUM

CSS-10 — Sticky Table Header on Scroll

EXPECTED APPROACH

```
<div class="max-h-96 overflow-y-auto">
  <table class="w-full">
    <thead class="sticky top-0 bg-white shadow">
      <tr><th class="p-2">Name</th><th class="p-2">Score</th></tr>
    </thead>
    <tbody>...</tbody>
  </table>
</div>
```

WHAT TO CHECK

- sticky and top-0 applied to the thead (or its row).
- The scrollable ancestor keeps overflow-y-auto and a bounded height (max-h-96).
- A background color (bg-white or similar) is set on the sticky header.
- th padding and table structure otherwise unchanged.

COMMON MISTAKES

- Using fixed instead of sticky, detaching the header from the table's own scroll container entirely.
- Forgetting a background color, so scrolled body rows visually show through the header text.
- Applying sticky to the whole table element instead of the thead/tr.
- Removing the bounded height/overflow on the ancestor, leaving sticky with no scrolling context to stick within.

SUGGESTED SCORING

sticky + top-0 on header	40%
Scroll container setup preserved	30%
Background color fix for overlap	30%

CSS MEDIUM

CSS-11 — Responsive Typography for a Hero Heading

EXPECTED APPROACH

```
<h1 class="text-3xl sm:text-4xl lg:text-6xl font-bold max-w-2xl leading-tight">Build faster with our platform<
```

WHAT TO CHECK

- Base (mobile) size is smaller than the original text-5xl, e.g. text-3xl.
- sm: and lg: breakpoints increase the size in ascending order.
- A max-w-* utility constrains line length.
- A tighter leading (line-height) utility is considered appropriate for a large heading.

COMMON MISTAKES

- Breakpoint sizes not in ascending order (e.g. the lg: size accidentally smaller than the sm: size).
- Omitting the unprefixed base class, leaving mobile at the browser default size instead of an intentional smaller size.
- Missing a max-w-* constraint, causing very long unreadable lines on wide screens.
- Using px-based margin instead of a width constraint to try to limit line length.

SUGGESTED SCORING

Correct ascending responsive text sizes	50%
max-width constraint added	25%
Leading/line-height consideration	25%

CSS MEDIUM

CSS-12 — Consistent Vertical Spacing in a Form

EXPECTED APPROACH

```
<form class="space-y-4">
  <div><label>Name</label><input class="border w-full"/></div>
  <div><label>Email</label><input class="border w-full"/></div>
  <div><label>Password</label><input class="border w-full"/></div>
</form>
```

WHAT TO CHECK

- space-y-4 (or similar single value) applied to the form.
- Individual mb-1 / mb-5 classes removed from the children.
- The same spacing value is used consistently rather than mixed values.
- Candidate understands space-y-* only adds margin-top to non-first children.

COMMON MISTAKES

- Leaving the old mb-1/mb-5 classes on children alongside space-y-4, causing uneven doubled gaps.
- Applying space-y-4 to a flex-row container, where it has no visible effect without flex-col since it targets vertical stacking.
- Expecting space-y-4 to add padding instead of margin between siblings.
- Assuming space-y-4 also adds spacing above the first field or below the last.

SUGGESTED SCORING

Correct space-y-4 usage on parent	45%
Removal of conflicting per-child margins	35%
Understanding of sibling-only application	20%

CSS MEDIUM

CSS-13 — Bootstrap Responsive Column Layout

EXPECTED APPROACH

```
<div class="container">
  <div class="row g-3">
    <div class="col-12 col-md-6 col-lg-4"><div class="card">Product A</div></div>
    <div class="col-12 col-md-6 col-lg-4"><div class="card">Product B</div></div>
    <div class="col-12 col-md-6 col-lg-4"><div class="card">Product C</div></div>
  </div>
</div>
```

WHAT TO CHECK

- col-12 used as the mobile-first full-width base.
- col-md-6 produces 2 cards per row on medium screens.
- col-lg-4 produces 3 cards per row on large screens (12/4=3).
- row is wrapped inside a container, and a gutter class like g-3 is applied.

COMMON MISTAKES

- Using col-md-4 without a col-12 base, leaving mobile behavior undefined or relying on auto-sizing instead of explicit full width.
- Column math error, e.g. col-lg-3 giving 4 per row instead of the requested 3.
- Forgetting the .row wrapper around the .col elements, breaking the grid's negative-margin gutter system.
- Omitting the container, causing content to touch the viewport edges.

SUGGESTED SCORING

Correct column breakpoint math	55%
Row/container structure correct	25%
Gutter spacing applied	20%

CSS MEDIUM

CSS-14 — Flex Items Not Wrapping to a New Line

EXPECTED APPROACH

```
<div class="flex flex-wrap gap-2 w-80 border p-2">
  <span class="px-3 py-1 bg-gray-200 rounded-full">Design</span>
  <span class="px-3 py-1 bg-gray-200 rounded-full">Engineering</span>
  <span class="px-3 py-1 bg-gray-200 rounded-full">Marketing</span>
  <span class="px-3 py-1 bg-gray-200 rounded-full">Sales</span>
  <span class="px-3 py-1 bg-gray-200 rounded-full">Support</span>
</div>
```

WHAT TO CHECK

- flex-wrap added to the container.
- gap-2 (single gap utility) is recognized as covering spacing on both the row axis and between wrapped lines.
- w-80 and border are retained, keeping the container's bounded width.
- Candidate names the default flex-nowrap as the root cause.

COMMON MISTAKES

- Removing the width constraint instead of adding flex-wrap, defeating the purpose of a bounded container.
- Using flex-wrap-reverse instead of flex-wrap, unnecessarily reversing row order.
- Assuming gap doesn't apply to wrapped rows and manually adding extra margin classes instead.
- Not recognizing flex-nowrap as the default behavior causing the overflow in the first place.

SUGGESTED SCORING

flex-wrap added	50%
Gap spacing correct on both axes	30%
Container width/border preserved	20%

CSS MEDIUM

CSS-15 — Dashboard Layout with Named Grid Areas

EXPECTED APPROACH

```
.dashboard {
  display: grid;
  grid-template-columns: 220px 2fr 1fr;
  grid-template-rows: 60px 1fr 1fr;
  grid-template-areas:
    "sidebar topbar topbar"
    "sidebar main   stat1"
    "sidebar main   stat2";
  min-height: 100vh;
  gap: 1rem;
}
.sidebar { grid-area: sidebar; }
.topbar  { grid-area: topbar; }
.main    { grid-area: main; }
.stat1   { grid-area: stat1; }
.stat2   { grid-area: stat2; }
```

WHAT TO CHECK

- sidebar area name repeats in all three row strings, spanning the full grid height.
- topbar spans the remaining two columns in the first row.
- main area name repeats across rows 2 and 3 to span two rows.
- Each row string in grid-template-areas has exactly 3 tokens, matching grid-template-columns.

COMMON MISTAKES

- Row strings with inconsistent token counts across rows, invalidating the whole grid-template-areas rule.
- Forgetting to repeat 'sidebar' in every row, leaving it confined to a single cell.
- Not applying grid-area to one or more of the child elements.
- Confusing the order of grid-template-columns and grid-template-rows values.

SUGGESTED SCORING

Valid template-areas with consistent columns	40%
Correct area spans (sidebar full height, main 2 rows)	40%
Correct grid-area assignment on children	20%

CSS MEDIUM

CSS-16 — Uniform Aspect-Ratio Image Cards

EXPECTED APPROACH

```
<div class="grid grid-cols-3 gap-4">
  <img src="a.jpg" class="w-full aspect-square object-cover"/>
  <img src="b.jpg" class="w-full aspect-square object-cover"/>
  <img src="c.jpg" class="w-full aspect-square object-cover"/>
</div>
```

WHAT TO CHECK

- aspect-square applied to each image for a consistent 1:1 ratio.
- object-cover used so the image crops to fill the box without distortion.
- w-full retained so aspect-square has a width basis to compute height from.
- grid-cols-3 layout left unchanged.

COMMON MISTAKES

- Using object-contain instead of object-cover, leaving empty letterboxed space instead of filling the box.
- Dropping w-full, leaving aspect-square with no defined width to compute a height from.
- Setting a fixed h-* value instead of aspect-square, breaking consistency across responsive column widths.
- Forgetting object-cover entirely, leaving the image stretched/distorted to fit the square.

SUGGESTED SCORING

aspect-square applied correctly	40%
object-cover applied correctly	40%
Responsive width retained (no fixed height)	20%

CSS MEDIUM

CSS-17 — Modal Overlay Centering with Z-Index

EXPECTED APPROACH

```
.overlay {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 50;
}

.modal {
  background: white;
  padding: 2rem;
  border-radius: 8px;
  max-width: 400px;
  width: 90%;
}
```

WHAT TO CHECK

- overlay uses position: fixed with inset: 0 (or all four offsets set to 0) to cover the full viewport regardless of scroll.
- overlay is a flex container with align-items and justify-content set to center the modal.
- z-index is set high enough on the overlay to sit above normal page content.
- modal has a solid, contrasting background so it reads clearly against the dimmed backdrop.

COMMON MISTAKES

- Using position: absolute instead of fixed, tying the overlay to its nearest positioned ancestor instead of the viewport, breaking full-page coverage on scroll.
- Forgetting z-index, so the overlay renders behind other stacked page content.
- Trying to center the modal with margin: auto alone without a flex or defined height context, which fails vertically.
- Using an opaque background color instead of a semi-transparent rgba value for the dimming effect.

SUGGESTED SCORING

Fixed full-screen overlay with inset:0	35%
Flex centering of modal	35%
Correct z-index/stacking	30%

CSS MEDIUM

CSS-18 — Divided Settings List

EXPECTED APPROACH

```
<ul class="border rounded-lg divide-y divide-gray-200">
  <li class="p-4">Notifications</li>
  <li class="p-4">Privacy</li>
  <li class="p-4">Security</li>
</ul>
```

WHAT TO CHECK

- divide-y applied to the parent ul.
- A divide color class (e.g. divide-gray-200) is specified.
- The outer border and rounded-lg remain unchanged on the list.
- No manual border-b classes added to individual li elements.

COMMON MISTAKES

- Adding border-b to every li including the last one, creating an extra unwanted line that duplicates the outer border.
- Using divide-x instead of divide-y, which is the wrong axis for a vertically stacked list.
- Forgetting a divide color class, leaving dividers effectively invisible.
- Removing the outer border thinking divide-y replaces it.

SUGGESTED SCORING

divide-y on parent list	45%
Divide color specified	30%
No redundant per-item borders	25%

CSS MEDIUM

CSS-19 — Equal-Height Pricing Card Grid

EXPECTED APPROACH

```
<div class="row g-3">
  <div class="col-md-4 d-flex"><div class="card w-100"><div class="card-body">Short text</div></div></div>
  <div class="col-md-4 d-flex"><div class="card w-100"><div class="card-body">A much longer paragraph of text
  <div class="col-md-4 d-flex"><div class="card w-100"><div class="card-body">Medium length text</div></div></
</div>
```

WHAT TO CHECK

- Candidate relies on the fact that .row is already a flex container with default stretch behavior across its columns.
- d-flex is added to each column and w-100 to each card so the card fills the stretched column's full height and width.
- g-3 gutter class retained for spacing.
- No fixed pixel height applied to the cards.

COMMON MISTAKES

- Setting a fixed height (e.g. height: 300px) on the cards, which breaks on longer text or different screen sizes.
- Not realizing Bootstrap rows are already flex containers, and adding unnecessary float or clearfix hacks instead.
- Forgetting w-100 on the card, so it doesn't fill the stretched flex column's width and height.
- Applying d-flex to the card itself instead of the surrounding column.

SUGGESTED SCORING

Correct use of row's flex-stretch behavior	50%
Card fills column via d-flex + w-100	30%
No fixed-height hack used	20%

CSS MEDIUM

CSS-20 — Equal Height Columns Without Explicit Heights

EXPECTED APPROACH

```
.columns {
  display: flex;
}

.col-a, .col-b, .col-c {
  flex: 1;
  padding: 1rem;
}
```

WHAT TO CHECK

- Parent .columns set to display: flex.
- Columns given flex: 1 to share width equally.
- No explicit height or height:100% needed — candidate relies on (or explicitly restates) the default align-items: stretch behavior.
- Candidate can explain why flex containers equalize child height by default in the row direction.

COMMON MISTAKES

- Using float: left on the columns, a classic pre-flexbox pattern that does not equalize height at all.
- Explicitly setting align-items: flex-start, which overrides the helpful default stretch behavior and reintroduces uneven heights.
- Hardcoding a fixed height value instead of relying on flex's stretch default.
- Forgetting flex: 1, leaving columns sized only to their content width instead of sharing the row evenly.

SUGGESTED SCORING

display:flex on parent	40%
flex:1 for equal width distribution	30%
Correct reasoning about the stretch default	30%

CSS MEDIUM

CSS-21 — Container Not Centering on Wide Screens

EXPECTED APPROACH

```
<div class="max-w-4xl mx-auto px-4">
  <h1>Welcome</h1>
  <p>Section content goes here.</p>
</div>
```

WHAT TO CHECK

- mx-auto added to the div.
- max-w-4xl retained, since mx-auto has no visible centering effect on a full-width block.
- px-4 inner padding retained.
- Candidate explains margin:auto centering requires a constrained width to have any visible effect.

COMMON MISTAKES

- Adding justify-center or items-center, which only work inside a flex or grid parent, not on a plain block-level div by itself.
- Removing max-w-4xl, leaving the div full-width with no visible effect from mx-auto.
- Using text-center, which centers inline content/text but not the block container itself.
- Wrapping in a new flex parent instead of using the simpler mx-auto fix.

SUGGESTED SCORING

mx-auto added correctly	50%
Understanding of the max-width prerequisite	30%
Padding retained	20%

CSS MEDIUM

CSS-22 — Auto-Responsive Card Grid Without Media Queries

EXPECTED APPROACH

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

WHAT TO CHECK

- Uses repeat() together with auto-fit (or auto-fill).
- minmax(220px, 1fr) sets the minimum card width and lets columns grow to fill remaining space.
- gap property set for spacing between cards.
- No media queries used, satisfying the fluid/no-breakpoint requirement.

COMMON MISTAKES

- Using a fixed repeat(4, 1fr) with manual media queries to change the column count at each breakpoint, which works but does not satisfy the no-media-query fluid requirement.
- Using auto-fill instead of auto-fit when the design wants existing items to stretch and fill leftover space rather than leaving invisible empty tracks.
- Omitting the minmax lower bound, letting columns shrink below a readable width on narrow screens.
- Using percentage-based fixed columns instead of minmax, losing the auto column count adjustment.

SUGGESTED SCORING

Correct repeat(auto-fit, minmax()) syntax	55%
Appropriate min/max values	25%
Gap and no-media-query requirement met	20%

CSS MEDIUM

CSS-23 — Aligned Label-Input Form Rows

EXPECTED APPROACH

```
<form class="space-y-2">
  <div class="flex gap-2"><label class="w-32 shrink-0">Name</label><input class="border flex-1"/></div>
  <div class="flex gap-2"><label class="w-32 shrink-0">Email Address</label><input class="border flex-1"/></di
  <div class="flex gap-2"><label class="w-32 shrink-0">Phone</label><input class="border flex-1"/></div>
</form>
```

WHAT TO CHECK

- A fixed width (e.g. w-32) is applied consistently to every label.
- shrink-0 added on each label so long text like 'Email Address' does not shrink below the fixed width.
- flex-1 retained on each input to fill the remaining row width.
- The same width value is used across all rows for true alignment.

COMMON MISTAKES

- Using different width values per label row instead of one consistent value across all rows.
- Forgetting shrink-0, so the longer 'Email Address' label shrinks below its intended width, breaking alignment.
- Setting the width on the input instead of the label.
- Using min-w-32 instead of w-32, allowing labels to still vary in width based on content.

SUGGESTED SCORING

Consistent fixed label width across rows	45%
shrink-0 to prevent label shrinking	30%
Input flex-1 retained	25%

CSS MEDIUM

CSS-24 — Multi-Line Text Truncation for Card Descriptions

EXPECTED APPROACH

```
<div class="grid grid-cols-3 gap-4">
  <div class="p-4 border rounded">
    <h3 class="font-bold">Title</h3>
    <p class="line-clamp-3">A long description that could run to many lines and break the card grid's uniform
  </div>
</div>
```

WHAT TO CHECK

- line-clamp-3 applied to the description paragraph.
- Candidate explains truncate only clamps to a single line and is insufficient here.
- grid-cols-3 gap-4 layout left unchanged.
- Ellipsis behavior is expected to appear at the 3rd line cutoff.

COMMON MISTAKES

- Using truncate instead of line-clamp-3, which only handles single-line clamping via white-space:nowrap.
- Using overflow-hidden alone without line-clamp-3, which hides overflow abruptly with no ellipsis and no reliable line count.
- Picking an arbitrary max-height in pixels instead of line-clamp-3, which doesn't guarantee a whole-line cutoff or ellipsis.
- Forgetting that line-clamp requires the text to actually be a block of wrapped text, not a single inline span.

SUGGESTED SCORING

Correct line-clamp-3 usage	55%
Correct reasoning versus single-line truncate	25%
Surrounding layout preserved	20%

CSS MEDIUM

CSS-25 — Bootstrap Toolbar Alignment

EXPECTED APPROACH

```
<div class="toolbar p-2 border-bottom d-flex justify-content-between align-items-center">
  <h5 class="mb-0">Documents</h5>
  <div class="d-flex gap-2">
    <button class="btn btn-primary">New</button>
    <button class="btn btn-outline-secondary">Import</button>
  </div>
</div>
```

WHAT TO CHECK

- d-flex applied to the outer toolbar.
- justify-content-between used to push the title and the button group to opposite ends.
- align-items-center used for vertical alignment.
- The two buttons are wrapped in their own d-flex gap-2 sub-container so they stay grouped together on the right.

COMMON MISTAKES

- Applying justify-content-between directly across all three items (title, btn, btn) instead of grouping the buttons, which spreads all three across the row instead of keeping the buttons together.
- Forgetting align-items-center, leaving the title and buttons misaligned vertically.
- Forgetting mb-0 on the heading, leaving default heading margin that throws off vertical centering.
- Using justify-content-end instead of justify-content-between, which pushes the title to the right along with the buttons.

SUGGESTED SCORING

d-flex + justify-content-between on outer toolbar	40%
Button grouping in its own sub-flex container	35%
align-items-center for vertical alignment	25%

CSS MEDIUM

CSS-26 — Button with Dark Mode and Focus States

EXPECTED APPROACH

```
<button class="px-4 py-2 rounded bg-blue-600 text-white hover:bg-blue-700 focus-visible:ring-2 focus-visible:r
  Save
</button>
```

WHAT TO CHECK

- dark: prefix used for a dark-mode-specific background color.
- hover: prefix used for a distinct shade on hover, different from the base color.
- focus-visible: (not plain focus:) used for the keyboard-only focus ring, with a ring utility.
- Base (unprefixed) classes remain as the default light-mode appearance.

COMMON MISTAKES

- Using focus: instead of focus-visible:, which shows the ring on every mouse click too, not just keyboard navigation.
- Forgetting the dark: prefix entirely and using one universal color that doesn't adapt to dark mode.
- Writing hover:dark:bg-blue-400 instead of dark:hover:bg-blue-400, getting the variant stacking order backwards.
- Omitting ring color/width utilities alongside focus-visible:, leaving an invisible or default-only ring.

SUGGESTED SCORING

Correct dark: variant usage	35%
Correct hover: variant	30%
Correct focus-visible: ring for accessibility	35%

CSS MEDIUM

CSS-27 — Sticky Sidebar That Scrolls With Content

EXPECTED APPROACH

```
<div class="flex gap-8 items-start">
  <aside class="w-64 sticky top-4">Table of contents links here.</aside>
  <article class="flex-1">Long article content spanning many screens of scrolling.</article>
</div>
```

WHAT TO CHECK

- sticky applied to the aside.
- top-4 (or a similar top offset) provided as the stick threshold.
- items-start added on the flex parent, since without it the default items-stretch makes the aside's box span the full article height, leaving sticky with no room to move.
- article/content column layout otherwise unaffected.

COMMON MISTAKES

- Forgetting items-start on the flex container, so align-items:stretch makes the sidebar's box already span the full content height, making sticky appear to do nothing.
- Using fixed instead of sticky, detaching the sidebar from its column and risking overlap with content at different widths.
- Omitting a top-* offset value, which sticky positioning requires to actually activate.
- Applying sticky to the outer flex container instead of the aside element itself.

SUGGESTED SCORING

sticky + top offset on aside	45%
items-start on flex parent, correctly explained	35%
Article layout preserved	20%

CSS MEDIUM

CSS-28 — Featured Item Spanning Multiple Grid Cells

EXPECTED APPROACH

```
<div class="grid grid-cols-3 grid-rows-2 gap-4">
  <div class="col-span-2 row-span-2 border p-4">Featured Story</div>
  <div class="border p-4">Story 2</div>
  <div class="border p-4">Story 3</div>
  <div class="border p-4">Story 4</div>
  <div class="border p-4">Story 5</div>
</div>
```

WHAT TO CHECK

- col-span-2 and row-span-2 both applied to the featured item.
- grid-cols-3 retained on the parent.
- grid-rows-2 (or reasoning about implicit row sizing) considered so the row span has row tracks to span.
- The other story items are left without any span classes.

COMMON MISTAKES

- Applying row-span-2 without considering that items beyond the explicit rows will flow into an implicit row sized differently (auto) than the explicit ones.
- Forgetting col-span-2 and only setting row-span-2, making the story tall but not wide.
- Adding span classes to more than one item unintentionally, breaking the single-featured-item intent.
- Using col-start/col-end line numbers inconsistently instead of the simpler col-span-2 utility when a span was all that was needed.

SUGGESTED SCORING

Correct col-span-2 row-span-2 on featured item	55%
grid-cols-3 base retained	25%
Other items left unspanned	20%

CSS MEDIUM

CSS-29 — Navbar Items Misaligned Despite justify-between

EXPECTED APPROACH

```
<nav class="flex justify-between items-center p-4 border-b">
  <img src="logo.svg" class="h-8"/>
  <div class="flex gap-4">
    <a href="#">Home</a>
    <a href="#">Pricing</a>
    <a href="#">Contact</a>
  </div>
</nav>
```

WHAT TO CHECK

- items-center added to fix the cross-axis (vertical) alignment.
- justify-between retained unchanged for horizontal distribution.
- Candidate explains justify- controls the main axis while items- controls the cross axis in a flex row.
- The inner nav-link group's own flex gap-4 remains unaffected.

COMMON MISTAKES

- Replacing justify-between with items-center instead of adding both, losing the intended left/right spacing between logo and links.
- Trying to fix the misalignment with margin/padding tweaks on the image instead of the correct cross-axis alignment utility.
- Confusing align-content with align-items — align-content affects multi-line/wrapped flex containers, not a single-line row like this one.
- Adding items-center only to the inner link group instead of the outer nav, which does not fix the logo's alignment.

SUGGESTED SCORING

items-center added correctly	50%
justify-between preserved	25%
Correct axis reasoning explained	25%

CSS MEDIUM

CSS-30 — Responsive Padding for a Hero Section

EXPECTED APPROACH

```
<section class="px-4 py-12 sm:px-8 sm:py-16 lg:px-16 lg:py-24 bg-indigo-600 text-white">
  <h1>Welcome to our platform</h1>
  <p>Get started in minutes.</p>
</section>
```

WHAT TO CHECK

- Horizontal padding increases across breakpoints (px-4 base, larger sm:/lg: values).
- Vertical padding starts smaller on mobile (py-12) and increases at sm/lg rather than being one large fixed value everywhere.
- Breakpoint values are in ascending order at each successive prefix.
- Base (unprefixed, mobile) classes are present for both axes.

COMMON MISTAKES

- Only adjusting horizontal padding and leaving py-24 unprefixed at every screen size, leaving the original mobile cramped/imbalanced complaint only partly fixed.
- Breakpoint values not in ascending order, e.g. an lg: value smaller than the sm: value.
- Using margin utilities instead of padding, when the scenario specifically describes internal breathing room within the section's background.
- Forgetting the base unprefixed classes and relying only on sm:/lg: prefixed values, leaving mobile unstyled.

SUGGESTED SCORING

Responsive horizontal padding scaling	35%
Responsive vertical padding scaling	35%
Ascending/mobile-first correctness	30%